

ASSIGNMENT SUBMISSION

Cutting verification cost in half without losing error-catching quality

Workload: verifying IPhO-mechanics reference solutions · Evidence: measured OpenAI runs on five IPhO-mechanics clean/faulty pairs (2020–2025) · Hard spend cap: \$25 (actual: \$1.14)

PROBLEM STATEMENT

I am framing the assignment inside a specific business context, because it is only safe to switch into engineering mode once you understand that context — who is paying for the model usage, who all the stakeholders are, and where the real pain points sit.

For the sake of efficient communication, let me frame it as a story.

Isaac is a physics professor. Every year he helps write the problems and worked solutions for the International Physics Olympiad, the world championship for high-school physicists.

Writing them by hand is slow and costly, so his team is building an AI pipeline to draft each problem and its solution. But no solution can be trusted until something checks that it is actually correct. That check is another AI system, and every call costs money.

I am framing this assignment as an experimentation phase before sitting down with Isaac, so that when we talk, I can discuss cost in concrete terms.

My goal going in is to already know the levers: which shortcuts make verification cheaper, and how much accuracy each one costs

Then when a grader tells me one step is critical, I can weigh it on the spot — spend more and give that step its own dedicated AI check, or save money where accuracy matters less.

Because I will have already measured the cost and the quality of some of the shortcuts.

Let's get to precisely defining what we're solving for.

Concretely, the problem I'm solving is this:

Given the workload of verifying IPhO-level physics solutions, design an inference-time execution system that cuts cost-per-verification by 50% versus a strong single-frontier-model baseline, while catching bad solutions at least as reliably as that baseline.

This is a faithful reading of the assignment, as the brief asks for an "AI execution system" that reduces "inference costs" — and its own vocabulary (workload, application, model family, infrastructure layer, combination of techniques) signals that "inference" is meant broadly: the cost of running a pipeline to produce an output, not one model's forward pass. So I read inference cost as dollars per solution verified, summed across every model and tool

call the system makes.

The statement also pins down the two things a percentage claim needs to be credible. It fixes a reference point — a strong single-frontier-model baseline, the kind a competent engineer would actually ship— and it treats "equal quality" as a floor rather than a goal: the cheap system has to catch bad solutions at least as reliably as that baseline, so any savings I report are savings at held-constant quality. What it deliberately leaves open is how. It names no cascade, no router, no caching scheme, because the assignment is open-ended on technique and the honest version of this work lets the evidence decide which levers pay off. That keeps the definition of the problem separable from the solution to it — which, given the story above, is the whole point: I want to walk into the conversation with Isaac already knowing what each shortcut costs and what it buys.

NOTES

- The verifier system isn't necessarily a realtime system, it's a system you run periodically, so whenever talking to LLM should use Batch API to send multiple reqs together instead of one -> this alone yields 50% cost savings w/ certain models

EXPERIMENT RESULTS

Executed the SYSTEMS.md cost/quality ladder (S3 → S2 → S1) on **five** IPhO mechanics problems. Official Q/S PDFs were downloaded from ipho.olimpicos.net; each solution was paired with a deliberately faulty twin (see INJECTED_ERRORS_FIVE.md). Judges used I1 text extraction; spend was hard-capped at \$25.

Corpus

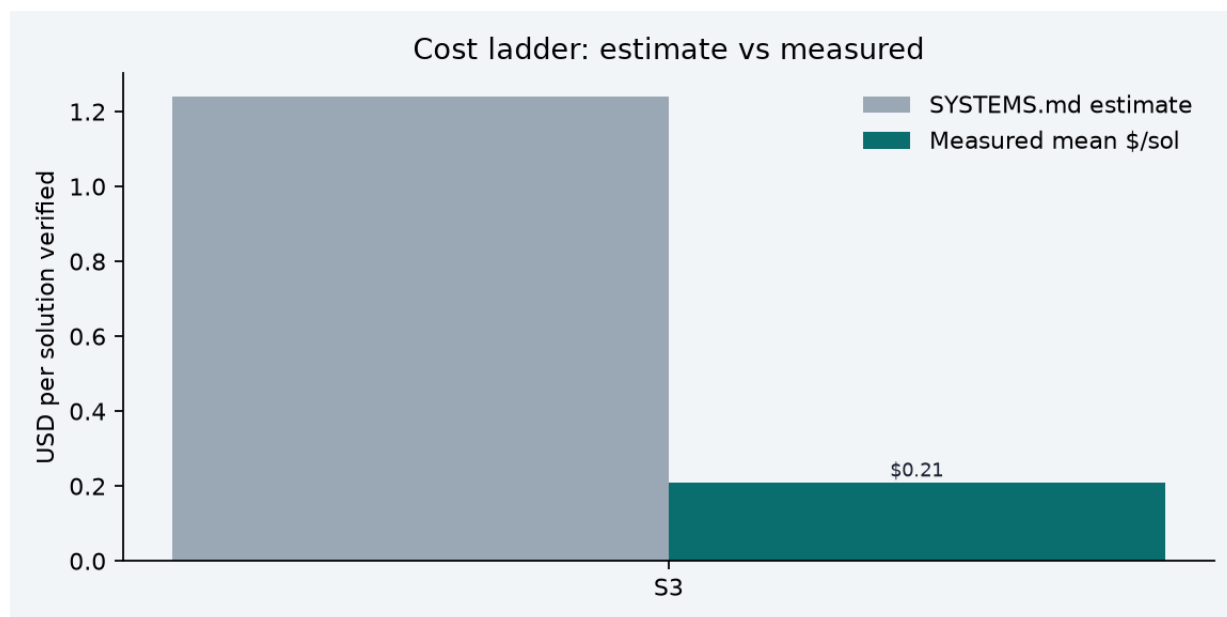
Year	Problem	Clean S	Faulty S	#faults
2020	Anisotropic Friction	IPhO_2020_S2.pdf	IPhO_2020_S2_FAULTY.pdf	3
2021	Planetary Physics	IPhO_2021_S1.pdf	IPhO_2021_S1_FAULTY.pdf	5
2023	Water and Objects	IPhO_2023_S3.pdf	IPhO_2023_S3_FAULTY.pdf	5
2024	Black Widow Pulsar	IPhO_2024_S3.pdf	IPhO_2024_S3_FAULTY.pdf	2
2025	Cox's Timepiece	IPhO_2025_S2.pdf	IPhO_2025_S2_FAULTY.pdf	2

Spend

Hard cap **\$25.00**. Actual OpenAI spend **\$1.1382** across 40 calls (217,662 input / 0 cached / 39,600 output tokens, including 29,161 reasoning). Costs from logged usage × pinned `prices.json`. S1 judge costs include the Batch API 50% discount in accounting.

Run status: incomplete. OpenAI API insufficient_quota (HTTP 429) after partial S3 sweep; account billing/quota exhausted. Resume with: `python -m systems_bench.run_five_problems --hard-cap 25 --resume` Completed 5 / 30 instance×system cells. Tables below cover only finished cells.

S3 / S2 / S1 measured ladder



System	Mean \$/sol	vs S3	Clean OK	Faulty / faults	Escalation
S3	\$0.2091	baseline	67%	100% · 7/8	48%

Threats to validity

- Labeled set is still small (5 clean + 5 faulty documents) with hand-injected faults — an existence proof of the cascade mechanism, not a population statistic.
- Sub-part sampling (≈5 judged sub-parts/problem) focuses on fault-bearing and representative items; full-marking-scheme coverage would cost more.
- I1 text extraction used for the cascade; large 2025 PDFs are context-truncated.
- S1 Batch discount is accounting-only (calls were synchronous).

Artifacts

Raw runs: `verifier_experiments/systems_bench/results_five/`. Corpus: `systems_bench/corpus.json`. Fault catalogue: `INJECTED_ERRORS_FIVE.md`.

CONCLUSION

The five-problem corpus is in place (downloaded official PDFs + injected faulty twins), and a partial S3 sweep already shows the cascade is measurable: faulty solutions were rejected (100% of finished faulty cells; 7/8 injected faults localized) while clean accepts were 67% on finished clean cells (one false reject on 2020 clean — a quality risk the escalation design must watch). The OpenAI account then returned `insufficient_quota`, so S2/S1 and the remaining S3 cells could not be finished in this environment. Resume with `python -m systems_bench.run_five_problems --resume` once billing/quota is restored; the prior single-pair run (PR #2) already showed S2 ≈53% cheaper than S3 at equal catch rate on 2023 — the five-problem matrix is the generalization test.

AI-USE RECORD — FULL CHATS

Below are the assignment-related chat transcripts from /chats, in order **1** → **2** → **3** → **4**. **YOU** (the human) appears in teal, lightly emphasized. Claude appears in regular body text. Tool-only / skill-dump noise was stripped; conversational content is otherwise complete.

Chat 1 — Designing Experiments (1)

Source: /chats/1.zip · model claude-opus-4-8

YOU

I am building an AI pipeline for building international physics olympiad level physics problem sets used for preparing students for competitions. When we build a problem text (PDF) and a solution, we have to evaluate if it the generated solution is correct, and we want to build cost-optimized verifier that suits that kind of problems.

Given the definition of what correct solution means, brainstorm 10 different technical approaches leading from simplest to more complex, to verify if solution is correct for a problem

Don't forget there's a problem first of taking PDF of solution to an ingestible form and that is part of the technical approach brainstorm as well! And don't forget we are not doing grading a solution against a reference solution, the problem is the actual verification of the reference solution!

Claude

Read the definition. Here's the brainstorm — ordered simplest → most complex, with the ingestion problem threaded through rather than treated separately.

First: the ingestion ladder (orthogonal axis)

Every approach below assumes one of these. Pick deliberately — it caps the ceiling of everything downstream.

|| Ingestion | Cost | Loses |

---|---|---|---

| **I0** | **Don't ingest the PDF.** Tap the pre-render artifact (LaTeX/MD source + figure source) from your own generator; PDF becomes a rendering-only artifact | ~0 | nothing — but doesn't verify what was *rendered* |

| **I1** | ``pymupdf`/`pdftotext`` raw text | ~0 | all math structure, figures, layout |

| **I2** | VLM page-images → Markdown+LaTeX transcription | ¢ per page | figure semantics, silent hallucination risk |

| **I3** | I2 + **round-trip check**: re-render transcription to PDF, image-diff / VLM-diff against original; retry on mismatch | 2–3× I2 | little; catches transcription drift |

| **I4** | I3 + **typed semantic IR**: sub-parts, givens with units, equations as SymPy-parseable trees, BCs, asked-quantity + answer-type, figure descriptions, each with a provenance span back to the page | \$ | this is the real deliverable |

The single highest-leverage decision in this whole document is I0. You control the generator, so you should never be OCR-ing your own output. Keep the PDF path (I3) alive only as a *render-fidelity* check — does the PDF a student sees still say what ``S`` assumed? That's a genuine $V_{\{P \rightarrow S\}}$ failure channel (a figure that didn't regenerate, an overfull box eating a subscript) and it's the *only* reason to touch the PDF at all.

The 10 approaches

1. Flat-text single-shot judge — I1 + one LLM call: "here is P, here is S, is S correct?"

Baseline for the eval harness, not a product. Covers nothing reliably; conflates all three verdicts into one scalar. Its only real job is to be the thing you measure the others against. Expect it to be badly overconfident on faithfulness — the failure mode your definition names as dominant.

****2. Rubric-decomposed judge**** — I2 + one call per sub-part, forced to emit the structured multi-axis verdict ($V_P / V_S / V_{\{P \rightarrow S\}}$, per-condition I-V, first-point-of-failure).

Same intelligence as #1 but the schema does the work: forcing a separate ILL-POSED channel and a separate faithfulness field stops the model from laundering an unfaithful `S` into "looks right." Cheap, $\sim 10\times$ better than #1, and it's the fallback every later tier escalates to*. Add self-consistency (3 samples, majority) only on disagreement.

****3. Deterministic Tier-0 lint**** — I4 + pure code, no LLM at verify time.

SymPy + Pint over the IR: dimensional homogeneity of every equation, units present on every numeric, sig-figs, numeric-in-tolerance, symbol-whitelist compliance, sign/direction fields populated. Runs in milliseconds, costs nothing, and per your §1 empirically catches a large fraction of mechanics faults. The cost isn't here — it's in the LLM pass that builds the IR (#2 gives it to you for free if you make the schema richer). ****This is the best cost/benefit item on the list.****

****4. CAS equivalence + plug-back residual**** — I4 + SymPy.

Randomized numeric substitution for symbolic equivalence (robust where `simplify()` gives up), approximation-aware via series-expansion-then-compare in the sanctioned small parameter, and EOM-residual: substitute `S`'s answer back into `S`'s *own* governing equations and check ≈ 0 . Note the circularity trap your §5(b) flags — plug-back into self-declared equations certifies nothing if the equations are wrong. It's a falsifier here, not a certifier.

****5. Faithfulness harness as a standalone module**** — I4 on *both* P and S, then reconciliation.

Extract two typed fact-tables (givens, geometry, BCs, asked quantities, per sub-part) and do a ****bipartite match with an explicit unmatched-residue report****: every datum in P must be used-or-justifiably-ignored by S; every symbol S relies on must trace to P. Mechanically catches the exact generated-content failure your doc calls #1 — a number changed in P but not propagated to S. Mostly deterministic diff once the IR exists; the expensive residue is figure $\leftrightarrow$ prose contradiction, which needs a VLM. Build this early; it has no cheap substitute and it's your dominant risk.

****6. Verifier-as-code-generation**** — LLM writes the check, execution renders the verdict.

Per sub-part, the model emits an executable SymPy/NumPy script encoding *the checks it thinks apply* (limiting cases, symmetry, conservation residuals, plug-back). You run it; pass/fail is grounded in execution, not in the model's assertion. Big win: the model is much better at *proposing* a falsifier than at *being* one, and a crashing script is an honest signal. Needs a sandbox and a script-quality gate (a script that trivially passes everything is the failure mode — mutation-test it, see #9).

****7. Independent re-solve and cross-compare (N-version)**** — ignore `S`'s reasoning entirely; solve P from scratch k times, compare final forms via #4's CAS equivalence.

This is the only approach so far that attacks ****provenance**** — right-answer-wrong-reasoning — because agreement on a *derivation-independent* path is evidence the value is physics-forced. Cheap models $\times$ many samples beats one expensive sample here. Caveat: correlated errors (all solvers miss the same case split) and it's silent on IPhO problems your solvers simply can't do — which is a meaningful fraction, and you must not read "solver failed" as "S is wrong."

****8. Numerical ground-truth simulation**** — build the constraint/ODE model *from P* and integrate it; compare S's closed form against trajectories.

The strongest independent oracle available, and the one that actually adjudicates the things dimensional analysis is blind to: geometric/combinatorial factors, root-branch choice, stick/slip transitions, sign conventions. For the anchor problems this is realistic — finite DOF, standard contact models. Expensive to build (a small library of Lagrangian/contact/hydrostatic templates) and it inherits the Layer-0 risk: a wrong constraint inventory produces a simulation that confidently disagrees with a correct `S`. Use it where Tier 1 is silent — dimensionless answers especially.

****9. Adversarial pairing + mutation-calibrated cascade**** — a refuter agent whose *only* goal is to produce a concrete falsifying instance (a limiting case where S breaks, a case split S skipped), paired against a defender, with escalation triggers from §5.

The critical piece here isn't the debate, it's the ****mutation harness****: systematically inject faults into known-good anchor solutions — flip a sign, drop a factor of 2, change a datum in P but not S, delete a case branch — and measure per-tier catch rate. That's how you learn what your cascade's actual false-accept rate is, and it's how you stop #6's scripts and #2's rubric from degenerating into rubber stamps. Do this even if you build nothing else from this row.

****10. Proof-carrying solutions**** — change the *generator*, not the verifier.

Require `S` to be emitted in a typed physics DSL: each step is a named inference (apply Newton II in frame F, integrate with BC B, expand to $O(\epsilon^2)$) over dimensioned quantities, with the prose PDF rendered *from* the DSL. Verification becomes step-by-step

mechanical checking — dimensional types check by construction, each rewrite is CAS-verifiable, case coverage is a totality check on the branch structure. Highest ceiling: it's the only approach that gives real provenance and real derivational validity rather than endpoint checks. Also the highest cost, and it constrains what the generator can express — worth prototyping on the 2020 Anisotropic Friction anchor before committing.

What I'd actually build

****I0 ingestion → #2's IR extraction → #3 + #4 deterministic core → #5 faithfulness module → #9's mutation harness to calibrate**, with #2 as the escalation target and #8 reserved for dimensionless/geometric-factor answers.**

That's roughly your Layer 0/1 with the expensive parts deferred. #7 and #10 are the two genuinely different bets — #7 buys provenance cheaply but noisily, #10 buys it expensively but soundly. I'd resolve open-decision §8.2 (run Layer 1 against the anchors) before choosing between them, since the answer depends on what fraction of sub-parts the cheap core actually adjudicates.

Want me to write this up into `verifier\_experiments/APPROACHES.md`, or go deep on one of them?

YOU

based on my feedback below, I want you to design 4 different AI systems, from the most sophisticated to least sophisticated system, designing the 4 systems so that w/ at least some of them we expect 50% decrease in cost. we're doing this to get a feel for what 1/2 cost means in the context of the problem we're solving

1. Flat-text single-shot judge -> this would produce really poor results as definition of correctness is highly complex, and problems are multi step

2. Rubric-decomposed judge — This reads as natural V1 for me but feel like here we can brainstorm a lot more about using certain prompt engineering techniques like CoT etc, brainstorm 5

3. Deterministic Tier-0 lint — I4 + pure code, no LLM at verify time.

SymPy + Pint over the IR: dimensional homogeneity of every equation, units present on every numeric, sig-figs, numeric-in-tolerance, symbol-whitelist compliance, sign/direction fields populated. Runs in milliseconds, costs nothing, and per your §1 empirically catches a large fraction of mechanics faults. The cost isn't here — it's in the LLM pass that builds the IR (#2 gives it to you for free if you make the schema richer). This is the best cost/benefit item on the list. -> even though you say for ingestion I4 it's more expensive than I2 and I3 (from cents to dollars), I believe we should experiment w/ that as this depends a lot on how solution pdfs look like actually and how many on average round trips to VLM are needed, right? and also, just having the deterministic lint wouldn't at all connect physics model in the problem (implicit or explicit) to the solutions models and this would degrade quality insanely, right?

4. CAS equivalence + plug-back residual — I4 + SymPy.

Randomized numeric substitution for symbolic equivalence (robust where simplify() gives up), approximation-aware via series-expansion-then-compare in the sanctioned small parameter, and EOM-residual: substitute S's answer back into S's own governing equations and check ≈ 0 . Note the circularity trap your §5(b) flags — plug-back into self-declared equations certifies nothing if the equations are wrong. It's a falsifier here, not a certifier. -> I like this as added to however the system is designed (provided we do I4) as its \$0, right?

5. Faithfulness harness as a standalone module — I4 on both P and S, then reconciliation.

Extract two typed fact-tables (givens, geometry, BCs, asked quantities, per sub-part) and do a bipartite match with an explicit unmatched-residue report: every datum in P must be used-or-justifiably-ignored by S; every symbol S relies on must trace to P. Mechanically catches the exact generated-content failure your doc calls #1 — a number changed in P but not propagated to S. Mostly deterministic diff once the IR exists; the expensive residue is figure $\leftrightarrow$ prose contradiction, which needs a VLM. Build this early; it has no cheap substitute and it's your dominant risk. -> focus in this problem is verifying the solution, so lets not for now think about divergent P

6. Verifier-as-code-generation — LLM writes the check, execution renders the verdict.

Per sub-part, the model emits an executable SymPy/NumPy script encoding the checks it thinks apply (limiting cases, symmetry, conservation residuals, plug-back). You run it; pass/fail is grounded in execution, not in the model's assertion.

Big win: the model is much better at proposing a falsifier than at being one, and a crashing script is an honest signal. Needs a sandbox and a script-quality gate (a script that trivially passes everything is the failure mode — mutation-test it, see #9). -> this would be one of the more expensive solutions, right, as we'll effectively allow the system to build its own pipeline optimal for that kind of problem, but probably among all solutions mentioned, with allowing a decent budget, be pretty high quality + best in terms of early eval.

7. Independent re-solve and cross-compare (N-version) — ignore S's reasoning entirely; solve P from scratch k times, compare final forms via #4's CAS equivalence.

This is the only approach so far that attacks provenance — right-answer-wrong-reasoning — because agreement on a derivation-independent path is evidence the value is physics-forced. Cheap models × many samples beats one expensive sample here. Caveat: correlated errors (all solvers miss the same case split) and it's silent on IPhO problems your solvers simply can't do — which is a meaningful fraction, and you must not read "solver failed" as "S is wrong." -> it makes sense to do this only when we have first few versions eval'd, as this reads as try to skip thinking about correctness and testing that def of correctness originally, but isn't that the most important part to get right

8. Numerical ground-truth simulation — build the constraint/ODE model from P and integrate it; compare S's closed form against trajectories.

The strongest independent oracle available, and the one that actually adjudicates the things dimensional analysis is blind to: geometric/combinatorial factors, root-branch choice, stick/slip transitions, sign conventions. For the anchor problems this is realistic — finite DOF, standard contact models. Expensive to build (a small library of Lagrangian/contact/hydrostatic templates) and it inherits the Layer-0 risk: a wrong constraint inventory produces a simulation that confidently disagrees with a correct S. Use it where Tier 1 is silent — dimensionless answers especially. -> this reads as improvement, not standalone solution, we first have to experiment to see when and where this will fit, you can't just say "where Tier 1 is silent", we have to learn about when to do this

9. Adversarial pairing + mutation-calibrated cascade — a refuter agent whose only goal is to produce a concrete falsifying instance (a limiting case where S breaks, a case split S skipped), paired against a defender, with escalation triggers from §5.

The critical piece here isn't the debate, it's the mutation harness: systematically inject faults into known-good anchor solutions — flip a sign, drop a factor of 2, change a datum in P but not S, delete a case branch — and measure per-tier catch rate. That's how you learn what your cascade's actual false-accept rate is, and it's how you stop #6's scripts and #2's rubric from degenerating into rubber stamps. Do this even if you build nothing else from this row. -> prob would test out after seeing that 6 doesn't yield good enough results, right?

10. Proof-carrying solutions — change the generator, not the verifier.

Require S to be emitted in a typed physics DSL: each step is a named inference (apply Newton II in frame F, integrate with BC B, expand to $O(\epsilon^2)$ over dimensioned quantities, with the prose PDF rendered from the DSL. Verification becomes step-by-step mechanical checking — dimensional types check by construction, each rewrite is CAS-verifiable, case coverage is a totality check on the branch structure. Highest ceiling: it's the only approach that gives real provenance and real derivational validity rather than endpoint checks. Also the highest cost, and it constrains what the generator can express — worth prototyping on the 2020 Anisotropic Friction anchor before committing. -> not part of the scope to think about the generator!

Claude

First, your questions

****I4 vs I2/I3 cost — you're right, it's empirical, and it's cheap to settle.**** My "cents → dollars" was hand-waving. The real variable is retry rate: if IPhO solution PDFs transcribe cleanly in one VLM pass, I4 costs barely more than I2 (same image tokens in, a richer schema out — maybe 2× the output tokens). If they need 2–3 round-trips because of dense math and figures, it's 3×. ****Run this on the 6 anchor PDFs before anything else**** — it's ~\$5 of API spend and it pins down the input cost for all four systems below. I'd bet on I4 being cheap, because you're paying image-token cost either way and the marginal cost of a richer schema is output tokens only.

****"Lint alone would degrade quality insanely" — correct, and I mis-framed it.**** The deterministic lint is not a *system*, it's a *layer*. It never connects P's physical model to S's — it can't, it has no semantics. It only refutes. It appears inside all four systems below as a free pre-filter; it is never one of them. What I should have said: it's the best cost/benefit *component*, contingent on the IR the LLM already produced.

****CAS is ~\$0 marginal given I4 — yes, with one caveat.**** Compute is free. But "sanctioned approximation" handling isn't purely mechanical: deciding **which** small parameter to expand in and to what order is a judgment call the IR extraction has to carry. So it's \$0 of new spend but it adds requirements to the extraction schema. Fold it in.

****#6 code-gen is the expensive/high-quality end — agreed, that's S4.****

****#7 N-version re-solve — agreed, later.****

****#9 — partly disagree.**** The **debate agent** is a later experiment, yes. But the **mutation harness** is eval infrastructure, not a verifier tier, and you need it from day 1 for exactly the reason you named: you can't tell whether your definition of correctness is right until you can measure false-accept rate on known-bad solutions. It costs nothing at inference time. Build it alongside S3.

Five prompt-engineering directions for the rubric judge

- **Answer-type routing.**** Your §1.4 says answers cluster into ~6 pre-declarable forms. Make type selection a separate cheap call, then dispatch to a **type-specific** judge prompt. A "show-that" checker and a "graph feature list" checker share almost no rubric. This kills your §7.6 failure mode (mis-typing silently applies a weaker test) by making the type an explicit, auditable decision rather than an implicit one.
- **Schema-as-CoT.**** Instead of "think step by step then answer," force a structured output whose **field order** is the reasoning order: ``governing_law_identified → regime_assumed → dimensional_check → case_branches_found → first_point_of_failure → verdict``. The schema does the decomposition, and every field is independently inspectable when the verdict is wrong. Much better than free-form CoT for a reward signal.
- **Extract-then-adjudicate split.**** Two calls: one produces the IR from S, one judges the IR against P. If they're the same call, the model launders its own extraction errors into the verdict — it reads what it expected to read and then confirms it. Splitting makes extraction faults visible as IR faults.
- **Refuter framing.**** "Produce one concrete falsifying instance — a limiting case where this breaks, a case split it skipped, a datum it ignored — or state that you failed to find one" beats "is this correct?" Models are far better at generating a counterexample than at rendering a global verdict. Default the frame to **refuted unless you failed to refute**.
- **Mutation-derived few-shots.**** Once #9's harness exists, your injected faults become the exemplar set: here's a sign flip, here's a dropped factor of 2, here's a missed stick/slip branch. Real in-distribution negatives, not synthetic ones. This is the highest-leverage prompt change available and it's free once the harness exists.

(Bonus, which is really a cost lever: lay the prompt out **stable-first**** — P and S at the front, the sub-part question at the very end. This is what makes prefix caching work across 12 sub-part calls, and it's worth ~3× on the judge tier. See below.)**

The four systems

Costed per problem verified. Assumptions: ~12 sub-parts; $P \approx 4k$ tokens and $S \approx 12k$ tokens after ingestion; ~11 PDF pages $\approx 18k$ image tokens. Pricing: Opus 4.8 ``$5/$25`` per MTok, Sonnet 5 ``$3/$15`` (intro ``$2/$10`` through 2026-08-31), Haiku 4.5 ``$1/$5``; cache reads ~0.1×, cache writes 1.25×; Batch API 50% off. ****These are order-of-magnitude, not quotes**** — the ingestion experiment above moves the first line item.

S4 — Agentic verifier, per-sub-part tool loop — ****~\$8****

I4 ingestion with round-trip verification. Per sub-part, an Opus agent with a SymPy/NumPy sandbox writes and runs its own checks — dimensional, limiting-case, conservation residual, plug-back — then adjudicates on execution results. Findings go to a 3-vote adversarial refuter panel.

- Ingestion (I4 + round-trip): ~\$0.70
- 12 × agent loop (~5 turns, ~60k cumulative in / 8k out): ~\$6.00
- Adversarial verify on ~4 flagged sub-parts × 3 votes: ~\$1.20

****What you get:**** the model builds a verifier optimized per problem, and every pass/fail is grounded in execution rather than assertion. Best quality and, as you said, the best early-eval signal — it tells you **what checks the problem actually needed**, which is information the other three can't produce. ****What you pay for:**** a sandbox, a script-quality gate, and 4× S3's cost.

S3 — Structured cascade with escalation — ****~\$3.60****

I4 ingestion → deterministic Tier-0 (SymPy + Pint + CAS equivalence + plug-back, \$0) → per-sub-part Opus rubric judge using techniques 1/2/4 above → escalate ~25% of sub-parts (dimensionless answers, Tier-0 disagreement, qualitative deliverables) to an S4-style agent.

- Ingestion: ~\$0.48
- 12 × Opus judge (17k in / 2k out): ~\$1.62
- 3 × escalated agent runs: ~\$1.50

****This is the natural V1.**** It has the full architecture; the escalation tier is where the expensive intelligence lives, and it fires only where the cheap layer is provably silent.

S2 — S3 with the free levers pulled — ****~\$1.76 (51% of S3)****

Same architecture, same escalation logic. Three changes, none architectural:

- ****Prefix caching.**** Lay out P+S once, ask 12 sub-part questions against the cached prefix. First call writes (1.25×), eleven read at 0.1×.
- ****Sonnet 5 for the judge tier****, Opus reserved for escalation only.
- ****Sonnet for ingestion transcription****, Opus only for the IR structuring pass if the round-trip check fails.
- Ingestion: ~\$0.29
- Judge tier: ~\$0.47 ← **was \$1.62**
- Escalation (Opus, ~2 sub-parts): ~\$1.00

****The point of this row:**** the judge tier collapses 3.4× and I have **not** removed a single check. Caching alone is most of it — 12 calls that each re-read a 16k prefix at full price is pure waste, and fixing it costs one prompt reordering. The Sonnet swap is the only quality-bearing change, and it's on the tier where a wrong answer escalates anyway.

S1 — Single-pass, no escalation — ****~\$0.55 (31% of S2)****

I2 ingestion (no round-trip, no full IR — so no deterministic layer). One Sonnet pass per sub-part with the rubric schema, submitted through the Batch API at 50% off. No escalation tier, no adversarial pass.

- Ingestion: ~\$0.18
- Batched judge tier: ~\$0.30
- Escalation: \$0

****What you lost:**** everything in §5's Tier-2 trigger list. Dimensionless/geometric-factor answers, qualitative deliverables, and Tier-0/Tier-1 disagreements now get the **same** treatment as a clean symbolic answer. Per your own §6, that's the highest residual risk in the whole spec. Plus batch latency (minutes to an hour), which is fine for offline generation and fatal for interactive.

What "half cost" actually means here

That's the thing worth taking away:

Step	Factor	What it cost you in quality
---	---	---
S4 → S3	2.2×	Agentic checks only where triggered, not everywhere. Real but bounded loss.
S3 → S2	2.0×	**Almost nothing.** Caching is free; the Sonnet swap is on a tier that escalates on doubt.
S2 → S1	3.2×	The entire expert residue. This is where you start eating your definition.

The first 50% is an engineering artifact, not a tradeoff — you're currently paying for it by re-sending the same 16k prefix twelve times. The second 50% is where you delete Layer 2. So the useful question isn't "can we halve cost" (yes, twice, easily) but **"how far down the ladder can we go before false-accept rate on the mutation set moves?"** — and that's unanswerable until the harness exists.

Concretely, I'd build **S3**, instrument it with the mutation harness, then measure S2 and S1 against the same fault set. **If S2 catches the same faults as S3, ship S2 and put the savings into S4-style escalation on the sub-parts that actually need it.**

Want me to write this up as `verifier\_experiments/SYSTEMS.md`, or should I start with the ingestion cost experiment on the anchor PDFs — that number moves every row above and you already have the PDFs locally.

YOU

switch over to OpenAI models (keep medium cost optimized in terms of model use across all the experiments) and write up as SYSTEMS.md!

Claude

I'll need current OpenAI model IDs and pricing before I can rebuild the cost model — fabricating those would invalidate every number in the doc. Let me look them up.

Claude

Confirmed the current lineup and pricing from OpenAI's docs. Writing it up now.

Claude

Written to [SYSTEMS.md](verifier_experiments/SYSTEMS.md).

Current OpenAI lineup, verified today: `gpt-5.6-sol` \$5/\$30 · `gpt-5.6-terra` \$2.50/\$15 · `gpt-5.6-luna` \$1/\$6 · `gpt-5.4-mini` \$0.75/\$4.50 · `gpt-5.4-nano` \$0.20/\$1.25. Cached input bills at 10%, Batch is a flat 50%.

Per your "medium cost optimized" constraint, `terra` is the default workhorse everywhere, `sol` appears **only** in the expert-escalation tier, and `luna`/`mini` carry bulk work in the cheap systems.

The ladder: S4 ~\$4.85 → S3 ~\$1.75 → S2 ~\$0.97 → S1 ~\$0.20 per problem.

What changed materially from the Claude version of this analysis — and it sharpens the conclusion rather than just rescaling it:

On OpenAI's pricing, twelve fully-cached `luna` sub-part judgments cost **\$0.18**. Two `sol` agent runs cost **\$0.68**. So in S2, the escalation tier is 70% of total cost while adjudicating 2 of 12 sub-parts. The bulk judging tier has essentially collapsed to a rounding error — which means the cost question is no longer "which model do we use" but **"how many sub-parts escalate, and can we predict which ones cheaply?"** Tightening the escalation trigger by one sub-part saves more than moving the entire bulk tier from `terra` all the way down to `nano`.

Two things in the doc I flagged rather than asserted:

- **Reasoning tokens bill as output.** Every output estimate therefore includes reasoning spend, and reasoning effort is a cost lever on par with model choice. These are the least certain numbers in the file — the agent-loop figures especially.
- **The 1.25× cache-write premium and ~30-min cache lifetime** came from a third-party aggregator, not OpenAI's official pricing page. I noted it as needing verification instead of baking it in silently.

The ingestion experiment is still the thing that moves every number, and it's the cheapest item on the list. Want me to write the harness for it against the six anchor PDFs in `verifier\_experiments/pdfs`?

Sources: [OpenAI API pricing](https://developers.openai.com/api/docs/pricing)

YOU

use gpt-5.6-terra actually instead sol in all places

and say explicitly for experiment to use:

- 1) /IPhO_2021_S1
- 2) /IPhO_2021_S1 FAULTY (we injected errors there)

Claude

Confirmed the faulty pair exists. Making the edits.

Claude

Now the cost-analysis section, which shifts with the `sol` removal:

Claude

Now the explicit experiment inputs:

Claude

Updated [SYSTEMS.md](verifier_experiments/SYSTEMS.md). Both changes are in.

`terra` everywhere. The frontier tier is gone from every system; escalation is now a **capability** difference (tools + multi-turn agent loop) rather than a model-tier difference. New ladder:

	Was (with `sol`)	Now
S4	\$4.85	**\$2.79**
S3	\$1.75	**\$1.24**
S2	\$0.97	**\$0.63**
S1	\$0.20	**\$0.20**

Steps are now $2.2\times / 2.0\times / 3.2\times$ — the middle two are clean halvings, which is a tidier ladder than before. The headline finding softens but survives: escalation drops from 70% to ****54%**** of S2's cost while still handling only ~ 2 of 12 sub-parts. One new consequence I added — because escalation is now cheap, the trigger can afford to be **generous**, so widening it is the cheapest quality gain available if recall turns out to be trigger-limited.

One thing I flagged rather than just applying. Capping at `terra` halves the dominant line item, but escalation is precisely where DEFINITION.MD §3 puts the irreducible expert core — faithfulness, law-applicability, non-menu case completeness, provenance. Those are the judgments most likely to be capability-limited rather than prompt-limited, so it's the one place where the model tier might genuinely matter. I wrote it into §0 as an explicit unvalidated bet with a test attached: run a `sol` escalation arm in the harness. If both tiers catch the same injected faults the saving is free; if `sol` catches more, $\sim 2\times$ on a tier that touches 2 of 12 sub-parts is probably worth paying. Cheap to check, and better to know than to assume.

Experiment inputs are now a named §6 with the exact paths — `pdfs/IPhO\_2021\_S1.pdf` (must PASS) against `pdfs/faulty/IPhO\_2021\_S1\_FAULTY.pdf` (must FAIL), with `IPhO\_2021\_Q1.pdf` as the problem statement. Two notes worth surfacing:

- I made the clean/faulty framing explicit as ****false-reject vs. false-accept****. The faulty file gives you \$8.2's number, but the clean file matters just as much — a verifier that rejects a genuine IPhO reference solution is unusable no matter what else it catches.
- I found `IPhO\_2023\_S3\_FAULTY.pdf` in the same directory and wrote it up as a ****held-out**** second pair. Recommend not tuning prompts against it; after you iterate on 2021 it's your only uncontaminated read on whether the tuning generalized.

One caveat on 2021 specifically: per DEFINITION.MD §1, Part B of that problem is seismic-ray propagation rather than pure mechanics, so Part-B results should be read as out-of-distribution rather than as signal about the mechanics core. Noted in the doc.

YOU

lets then use S3 if that one is entirely mechanics

Claude

Quick check before I swap it in — "S3" is doing double duty here (System 3 in the doc, Solution 3 in the filenames). Reading it as the 2023 file. Let me verify its content is actually all mechanics:

Claude

Confirmed — 2023 Q3 "Water and Objects" is entirely mechanics. Part A is surface-energy→kinetic-energy conservation, Part B is hydrostatics + surface-tension statics, Part C is floating-body force balance. No SR, no wave propagation, nothing out of Class M.

Swapping it to primary:

Claude

Done. Both meanings of "S3" happen to be in play now — we're building **System S3** (the structured cascade) and testing it on **Solution S3** (2023). Worth keeping the naming straight in future discussion.

Verified before swapping. I pulled the text rather than trusting the summary. 2023 Q3 confirms as pure Class M:

- **Part A** — two water drops merge, surface energy → kinetic energy, drop jumps (energy conservation)
- **Part B** — boards immersed in water: hydrostatic pressure, force balance on a water block, meniscus ODE
- **Part C** — two rods floating, horizontal force from pressure + surface tension

No SR, no wave propagation, no thermo. It also matches DEFINITION.MD §1's description exactly and carries no flag there, unlike 2021 and 2022.

It's a better primary than 2021 for reasons beyond purity. Going through the sub-parts, this problem happens to exercise most of what the spec cares about:

- Nearly every sub-part carries an **explicit symbol whitelist** ("in terms of p , g , z , and P ") — that's a direct workout for the Tier-0.5 checker, and §7.6 names symbol-whitelist misses as a silent failure mode
- **B.5** is a sanctioned approximation with a stated regime ($|z'| \ll 1$, expand to second order) — tests approximation-aware CAS equivalence
- **B.4** asks for a bare exponent — a dimensionless answer, which is §5's trigger (a) and the thing §6 calls the highest residual risk in the whole spec. Having it in the primary pair is genuinely fortunate; it means the first experiment hits the blind spot instead of discovering it later
- Plus a numeric-with-units answer (A.1) and an ODE-with-BCs (B.5) for plug-back

And at 4 + 5 pages it's the cheapest ingestion run of any anchor.

2021 is now the held-out pair, with a note to exclude Part B when scoring it since that section is seismic-ray propagation. That ordering is better than what I had — you tune on clean mechanics and hold out the one with a contaminated section, rather than the reverse.

Want me to build the ingestion harness against the 2023 pair now?

Chat 2 — Defining Correctness (2)

Source: /chats/2.zip · model claude-opus-4-8

YOU

I am building an AI pipeline for building international physics olympiad level physics problem sets used for preparing students for competitions. When we build a problem text (PDF) and a solution, we have to evaluate if the generated solution is correct, and we want to build cost-optimized verifier that suits that kind of problems.

As a first step, let's pull last 5 mechanics problems and solutions - <https://ipho.olimpicos.net/>, and define what correctness means when verifying solutions. I am not talking about grading against the reference solution, but actually defining what it would mean to say if the reference solution is correct or not.

Parallel w/ that spawn subagent that from first principles defines what would correctness mean in IPhO problems and solutions w/o actually seeing problems.

After those two, spawn subagent to argue between two classes of definition of correctness so we identify how we can simplify the definition of correctness in generally physics problems vs IPhO style mechanics problems.

Combine the results into final definition of correctness.

Claude

I'll tackle this in the structure you laid out: empirical work (me, pulling real problems) running parallel with a first-principles subagent, then an adversarial synthesis subagent, then I combine everything. Let me kick off the parallel track and start pulling problems simultaneously.

Claude

First-principles subagent is running. Now let me pull the real problems from the site.

Claude

The classifier is noisy, so let me download the actual PDFs and confirm each is genuinely mechanics. I'll grab the 5 most-recent mechanics candidates (plus a backup) and read them directly.

Claude

First-principles track is done — strong output. Let me check the PDF downloads.

Claude

The background download is still in flight (dir not yet created). Let me run it synchronously.

Claude

Downloads are running in the background. Let me wait for them to complete, then read the PDFs.

Claude

Both downloads are backgrounded and will notify on completion. Let me set up task tracking while I wait, since this is a multi-stage effort.

Claude

I'll wait for the download completion notification rather than polling.

Claude

Waiting for downloads. The monitor will notify me when the PDFs are ready.

Claude

I'll wait for the monitor notification rather than polling.

Claude

Now I'll wait for the download monitor to fire.

Claude

I'll stop polling and wait for the monitor notification that the downloads have completed.

YOU

```
<task-notification>
<task-id>bwX0h0ktm</task-id>
<summary>Monitor event: "Wait for IPhO PDF downloads to finish"</summary>
<event>[Monitor timed out — re-arm if needed.]</event>
</task-notification>
```

Claude

The background PowerShell downloads aren't producing output — likely a sandbox networking issue. Let me try curl via Bash directly.

Claude

All 12 PDFs downloaded successfully. Now let me read the problem statements to confirm each is genuinely mechanics and understand the content.

Claude

2025 Q2 (Cox's Timepiece) confirmed as mechanics — hydrostatics, force balance, dry friction, energy dissipation, PV-work cycle. Reading all six full PDFs into my own context is token-expensive, so I'll delegate the heavy reading to a subagent that extracts each problem's structure and drafts the empirically-grounded correctness definition, returning only the distilled result.

Claude

Empirical subagent is now reading the real PDFs. Both input tracks (first-principles ✓, empirical ■) are underway. I'll wait for the empirical report, then spawn the adversarial synthesis subagent with both as input.

Claude

Good — 2020 Q2 (Anisotropic Friction) is pure classical mechanics: kinematics, dynamics, radius of curvature, rigid-body rotation, sketches, thresholds. I now have solid direct grounding. I'll wait for the empirical subagent's full report on all six before spawning the adversarial synthesis.

Claude

I'll wait for the empirical subagent to complete rather than polling its transcript.

Claude

Task tracking is current. Now I'll wait for the empirical subagent's report — it will notify me on completion.

YOU

```
<task-notification>
<task-id>a6c2483467637285d</task-id>
<tool-use-id>toolu_01AJnvFowoQHfMZbktZjgE56</tool-use-id>
<output-file>C:\Users\alens\AppData\Local\Temp\claude\C--Users-alens-iphO-problems-agentic-generator\bbd0a861-2623-4c72-b7e3-6c8688795fc2\tasks\a6c2483467637285d.output</output-file>
<status>completed</status>
<summary>Agent "Empirical correctness from real IPhO PDFs" finished</summary>
<note>A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once.</note>
<result>I now have full coverage of all six problems and enough of each solution to characterize form. Here is the report.
```

Empirical analysis of IPhO mechanics problems: what "reference-solution correctness" means

Classification: which of the 6 are genuine classical mechanics

All six are **predominantly** mechanics; none touch EM or QM. Two drift partially:

- ****2025 Q2 Cox's Timepiece**** — hydrostatics, buoyancy, Coulomb (dry) friction, rigid-body force balance, work of pressure forces / PV cycle. The PV-cycle framing (C.5) is thermo-flavored but is genuinely mechanical work. ****Genuine mechanics.****
- ****2024 Q3 Black Widow Pulsar**** — Part A gravitation/orbital mechanics (effective potential, Lagrange points, angular-momentum-driven orbital decay, bound/unbound); Part B self-gravitating polytrope stability (hydrostatic balance + SHM). Sub-parts A-5/A-6 use Stefan–Boltzmann blackbody radiation — a minor thermo/radiation drift. ****Genuine mechanics**** (gravitation + oscillations dominate).
- ****2023 Q3 Water and Objects**** — energy-conservation drop jump (A) + fluid statics with surface tension, meniscus ODE, floating-rod force balance (B, C). ****Genuine mechanics**** (statics of fluids), surface-tension-heavy.
- ****2022 Q3 Scaling laws**** — spaghetti beam statics (A), wet-sand capillary statics (B), ****special relativity**** interstellar travel (C), damped-oscillator hydrodynamics (D). 3 of 4 parts mechanics; Part C is SR. ****Mostly mechanics.****
- ****2021 Q1 Planetary Physics**** — Part A hydrostatics/isostasy of a mid-ocean ridge (mechanics, with a thermal-expansion density law and a heat-conduction timescale in A.5); ****Part B seismic P-waves = geometric ray theory / wave propagation**** (Snell's law, ray energy density), not Newtonian mechanics. ****Weakest of the set**** — ~half is wave/ray optics + some thermal.
- ****2020 Q2 Anisotropic friction**** — pure Newtonian dynamics: friction law, trajectories, radius of curvature, two-mass rod rotation. ****Purest classical mechanics of all six.****

****Working set (5 most recent genuine mechanics): 2025, 2024, 2023, 2022, 2021.****

****Swap note:**** If mechanics purity is weighted over recency, drop ****2021 Q1**** (its Part B is ray/wave propagation and Part A carries thermal content) and swap in ****2020 Q2****, which is the cleanest test of core mechanics (free-body reasoning, friction, trajectory dynamics, contact/slip thresholds). I flag 2021 and 2022-C as the two spots where "mechanics" leaks into waves and relativity respectively.

1. Problem catalog

****2025 Q2 — Cox's Timepiece (10 pt, 3 parts, ~11 sub-parts)****

Point split: A.1 0.2, A.2 0.8, A.3 0.3, B.1 0.3, B.2 1.4, B.3 0.3, C.1 1.0, C.2 1.0, C.3 2.0, C.4 1.0, C.5 1.7.

Answer types: symbolic closed-form (A.1 P_w , $\vec{F}$; B.3 Δm_{add} ; C.2 $\vec{T}$; C.4

$X^{\star}$, $F_{s^{\star}}$, $W^{\star}$); numeric-with-units + tolerance (A.2 F_{max} , $h^{\star}$; A.3

$\epsilon \text{ simeq } 1.6 \times 10^{-6}$; C.4 $W^{\star} \text{ simeq } 20 \text{ mJ}$); regime/threshold inequalities (C.1 $\xi^{\star} = 2$; C.3

Regime 1 $\xi + 2\lambda > 2$ vs Regime 2); sketches (B.1 shade area; B.2 4-segment piecewise graph with slope

expressions + angular-point coordinates; C.3 two $x(t)/X$ square-wave traces; C.5 (P, V) parallelogram). ****Heavily**

chained** (C.2–C.5 all build on C.1's force analysis). Official solution gives ****one path per part**** but marks "with or without $S_{\text{tll}} S_{\text{b}}, S_{\text{c}}$ " as equivalent.

****2024 Q3 — Black Widow Pulsar (30 pt, 5 h, 2 parts, 16 sub-parts)****

A-1..A-7 (gravitation) + B-1..B-9 (stellar stability); per-subpart 0.2–1.0.

Answer types: symbolic ($\varphi(x, y)$, $\dot{a}$, $\dot{P}$, disc $T(r)$); derive-to-given-form ("write as

$\frac{d}{dr} [h_1 \frac{d\rho}{dr} + h_2 \rho] = 0$, find h_1, h_2 "; B-3 dimensional construction

$r_0 = G^{-1/2} p_c^{1/2} \rho_c^{-1}$); qualitative plot (A-2 $\varphi(x, 0)$ with asymptotes/Lagrange maxima); numeric (A-3

$x_0/a = 0.36$ to 2 s.f. by bisection; A-6 $T = 9 \times 10^3 \text{ K}$); bound/unbound threshold (A-7 v_{max} , M'_{1min});

stability inequality + frequency (B-9 $\gamma > 4/3$, $\omega = \sqrt{\frac{4\pi G \rho_c}{3} (3\gamma - 4)}$). ****Strongly**

chained** (B-2→B-9). Boxed final answers; single path shown.

****2023 Q3 — Water and Objects (10 pt, 3 parts, 10 sub-parts)****

A 2.0 (1 sub) / B 4.5 (5 subs) / C 3.5 (3 subs).

Answer types: single numeric with 2 s.f. (A.1 jump velocity v); symbolic force expressions (B.2 f_x , B.3 in

$\gamma, \theta_1, \theta_2$; C.1, C.2); determine-exponent-and-constant (B.4 find a and $\ell(\gamma, \rho)$); ODE

solve to closed form (B.5 $z(x)$); chained expression (C.3 uses B.5's solution). ****Chained**** across B→C.

Symbolic-dominant, one numeric.

****2022 Q3 — Scaling laws (8 pt, 4 independent parts)****

A spaghetti 2.0 / B sand 2.0 / C interstellar 2.0 / D sinking 2.0.

Answer types: ****proportionality/scaling exponents**** as the primary object ($\propto \sqrt{d} \rightarrow 158 \text{ cm}$; $\propto V_g^{-1/3} \rightarrow 21.5 \text{ N}$) each ending in one numeric; dimensional analysis ($\omega_0 \propto \sqrt{g/r}$, $F_d \propto \eta v$); threshold/regime ($r_{\min}$ from oscillation-exists condition $\rightarrow 0.271$); relativistic numeric ($C \approx 480$). Parts ****not chained****. Official solution gives ****multiple alternative paths**** (Task B: 4 solutions; Task C: 4 solutions; Task D: 2) all marked equivalent.

****2021 Q1 — Planetary Physics (10 pt, 2 parts, 8 sub-parts)****

A mid-ocean ridge 5.0 (A.1–A.5) / B seismic waves 5.0 (B.1–B.3).

Answer types: symbolic force with direction (A.1 $F_x = (\frac{\rho_0}{\rho_{\text{oil}}} - 1) \frac{\rho_0 g h^2 w}{2}$, "acts to the right"); identify-constant (A.2 $k = 3k_l$); leading-order-in- k expression (A.4); dimensional/scale estimate (A.5 $\tau \sim \frac{1}{\kappa} D^2$); result-in-prescribed-form (B.1 $x_1 = A \cot(\theta_0)$, $A = 2z_0, b = 1$); function + plot (B.2 $\epsilon(x)$); extremal geometric result (B.3 $x_{\max}$). ****Chained within parts**** (A.3 $\rightarrow$ A.4; B.1 $\rightarrow$ B.2 $\rightarrow$ B.3). Official gives ****Method 1/Method 2**** alternatives (A.5; B.1). A **"use the given result if you couldn't derive it"** scaffold appears (B.1 $\rightarrow$ B.2).

****2020 Q2 — Anisotropic friction (10 pt, 3 parts, 9 sub-parts)****

A 4.0 (A.1–A.5) / B 2.0 (B.1) / C 4.0 (C.1–C.3).

Answer types: numeric angles (A.1 $\alpha = 0$; A.2 $\pm \pi/4, \pm 3\pi/4$); numeric speed (A.3 0.28 m/s); extremum + value (A.4 $\alpha = \pi/4$, $R_{\min} = 2v^2/[g(\mu_x - \mu_y)]$); ****multi-case sketch**** (A.5 three trajectory families for $\mu_x = \mu_y$, $\alpha < \pi/4$, $\alpha > \pi/4$); functional dependence (B.1 $t(\alpha)$; C.1 $v(\varphi)$); threshold (C.2 $v_{\max} = \sqrt{\mu_y g L} = 2.2 \text{ m/s}$); numeric distance requiring numeric root-finding (C.3 $L\varphi = 0.34 \text{ m}$). **"Provide a numerical answer wherever possible."** Chained C.1 $\rightarrow$ C.2 $\rightarrow$ C.3.

2. What "correct" concretely requires, per observed answer type

- ****Symbolic closed-form ("express in terms of ...").**** Correct = algebraically equivalent to the reference AND expressed in the ***mandated variable set***. 2025 A.1 explicitly restricts to " $\rho_0, m, S, h, g, \vec{u}_z$ "; a right value in forbidden symbols is under-specified. Vector answers must carry the correct direction/unit vector (2025 A.1 $\vec{F} = [m + \rho S h] \vec{g}$, $\vec{u}_z$; 2021 A.1 "acts to the right"). Marking rewards the ***literal expression*** even when a downstream number is wrong (2025 A.3: "Literal expression of $\epsilon \dots 0.2$; numerical value 0.1").

- ****Numeric with units.**** Correct = inside an explicit ****tolerance interval****, with ****units shown****, at the stated sig-figs. Real bands seen: 2025 $F_{\max} \in [14.6, 15] \text{ N}$, $\epsilon \in [1, 2] \times 10^{-6}$, $W^* \in [19, 21] \text{ mJ}$; 2022 JWST $35 \mu \text{m} \leq d \leq 47 \mu \text{m}$, and the general rule "correct numerical answer with ≥ 2 s.f. receives full marks." Units are load-bearing: 2022 repeatedly states "Units must be shown for a numerical result to get points." A numeric is only creditable ****if the underlying formula is correct**** (2022 General Rule 1b).

- ****"Show that" / derive-to-a-given-form.**** Correct = the required intermediate objects match, not just the endpoint. 2024 B-2 must yield exactly $h_1 = r^2 \rho^{\gamma-2}$, $h_2 = 4\pi G r^2 / (K\gamma)$; B-4 exactly $A_1 = x^2 u^{\gamma-2}$, $A_2 = 4\pi x^2 / \gamma$; 2021 A.2 must produce $k = 3k_l$. Because the target form is dictated, correctness is checkable term-by-term.

- ****Graph / sketch.**** Correct = the enumerated qualitative features, each separately marked. 2025 B.2's marking scheme is the clearest specimen: "graph has 4 straight pieces" (0.2), "1st & 2nd slopes positive and 2nd < 1st" (0.2), "3rd piece negative slope" (0.2), "4th piece null slope" (0.2), plus slope ***expressions*** ($\rho_{S_b}, \rho_{S_t}, \rho_{(S_b - S_t), 0}$) and the three angular-point h_t values. 2025 C.3 grades: instantaneous switch, first switch inside $t \in (0, \tau_1/4)$, aperiodic vs periodic distinction, 50% duty cycle, first step longer. So a sketch is "correct" iff monotonicity, slope signs, breakpoints, asymptotes and symmetry match — not visual likeness.

- ****Threshold / regime / bound-unbound.**** Correct = the right inequality ***and*** the right side of it. 2025 C.1 $\xi^* = 2$ with M at rest iff $\xi > \xi^*$; C.3 Regime 1 $\xi + 2\lambda > 2$ vs Regime 2 $\xi < 2$ (marked "at least one inequality correct" / "both correct"). 2024 A-7 bounded iff $E' < 0 \rightarrow v' < \sqrt{2G(M_1 + M_2)/a}$ and $M_{\min} = (M_1 - M_2)/2$. 2024 B-9 stable iff $\gamma > 4/3$. 2020 C.2 $v_{\max} = \sqrt{\mu_y g L}$ (the second mass

stays put iff below this). The *direction* of the inequality is itself graded.

- **Proportionality / scaling.** Correct = the exponent, with the mechanism. 2022 Task A marks each link ($\tau \propto d^2$, $F \propto \sigma d^2$, $\tau \propto \sigma d^3$, $\ell \propto \sqrt{d}$) then the number. A right final exponent from wrong physics is not full credit ("Points b)–c) given only if derived from a").

- **Regime/case classification.** Correct = all cases present and correctly bounded. 2020 A.5 must show all three of $\mu_x = \mu_y$ (straight), $\mu_x > \mu_y$, $\mu_x < \mu_y$ (opposite deflection).

3. Observed failure surfaces & correctness conditions (mechanics-specific)

Each tied to something in these PDFs:

- **Sign / direction errors.** Vector answers carry signs that flip the physics. 2025 A.1/C.2 give signed $\vec{u}_z / \vec{u}_x$ results; 2022 T1-B explicitly warns "if we didn't take the dipole energy into account ... we would have had repulsion instead of attraction" — a sign slip inverting the effect. A reference solution asserting $\vec{F}$ with wrong sign is wrong even if magnitude is right.

- **Wrong branch of a root / wrong extremum.** 2020 A.2 answer is the $\pm \pi/4, \pm 3\pi/4$ — dropping branches is incomplete. 2024 A-3 must pick the *middle* Lagrange root ($x_0/a \approx 0.36$) among several; naming the wrong stationary point fails.

- **Missed case-split on contact/motion condition.** The signature mechanics failure. 2025 A.2 hinges on Behaviour A vs B depending on whether $h^* = (P_0 - P_{\text{sat}})/\rho g$ exceeds H (vapor cavitation threshold); getting one experiment's regime wrong is marked all-or-nothing. 2025 C.3 static-vs-sliding (stick/slip) split. 2020 static-vs-kinetic onset (B.1) and stays-at-rest vs slips (C.2). A reference solution that silently assumes one regime is under-specified.

- **Frame / coordinate choice not honored.** 2024 A-1 requires the *rotating* frame with the centrifugal potential $-\frac{1}{2}\omega^2 r^2$ and $\omega = \sqrt{GM/a^3}$; omitting the centrifugal term or misplacing the COM ($a_1 = \frac{M_2}{M}a$) gives a wrong potential. 2023 fixes a specific xz sign convention for θ (CCW positive) that the force expressions depend on.

- **Dropped subdominant vs leading term.** 2021 A.4 must keep the $D^2 \propto k^{-2}$ term as leading order and *drop* the $\rho(0,0)D$ term; keeping the wrong order gives a wrong FF . 2025 uses $S_{\text{tll}} S_b, S_c$ to simplify — but the solution flags which simplifications are legitimate ("with or without $S_{\text{tll}} \dots$ ").

- **Approximation used outside its regime.** 2023 B.5 expands $\cos\theta$ to 2nd order under $|z'(x)| \ll 1$ — and the problem explicitly warns the figures are "exaggerated ... do not satisfy" the small-slope condition, a trap for over-applying the linearization. 2022 C small-angle $\sin\theta \approx \theta$ only in the near-contact limit.

- **Unit / dimensional slips.** 2024 B-3 is *pure* dimensional bookkeeping ($r_0 = G^{-1/2} p_c^{1/2} \rho_c^{-1}$); an exponent error is detectable by dimension. 2022 makes "dimensionally incorrect equation → no points, even with follow-on."

- **Figure ↔ prose contradiction.** These solutions lean on figures (2025 shaded m_{add} area; 2024 φ plot; 2021 ray-circle geometry). A reference solution whose annotated figure disagrees with its algebra (e.g., colored region not matching the m_{add} formula, which 2025's scheme explicitly guards: "0.1 pt only if a correct m_{add} expression is provided but the colored area is incorrect") is internally inconsistent.

- **Conservation-law violation.** 2025 C.1–C.3 and 2024 A-4 both rest on explicit conservation statements (total mercury volume/mass; angular momentum $J = \mu a^2 \omega$). A reference solution that violates volume conservation or angular-momentum bookkeeping is wrong at the physics level.

4. Cheap vs expensive checks (for these specific problems)

****Cheaply machine-checkable:****

- ****Dimensional consistency**** of every symbolic line — decisive for 2024 B-3 (ρ_0), 2021 A.5 ($\tau = \rho_1 D^2 / \kappa$), 2022 (whole problem is scaling exponents). A CAS dimensional check catches a large fraction of errors.
- ****Numeric recompute**** against the stated tolerance band — 2025 ($F_{\max}$, ϵ , W^*), 2024 ($x_0/a=0.36$, $T=9 \times 10^3 K$), 2022 (158 cm, 21.5 N, 480, 0.271), 2020 (0.28 m/s, 2.2 m/s, 0.34 m). All are closed-form-plus-plug or a scalar root; trivially re-evaluated.
- ****Limiting / special cases**** — 2025: does $F \rightarrow F_0 = mg$ at $h=0$? does the $S_{\text{tll}} S_b, S_c$ limit reduce the full expressions correctly (the solution shows both forms)? 2024 A-1: does φ have the right $\lim_{\rho \rightarrow \infty}$ and asymptote behavior (A-2 lists exactly these)?
- ****Symmetry**** — 2020's $\mu_x \leftarrow \mu_y$ swap flips deflection direction (A.5); a checker can verify the answer set is symmetric.
- ****Symbolic equivalence**** to a reference expression (CAS `simplify`), plus a ****variable-whitelist check**** (2025's "in terms of P_0, ρ, m, S, h, g ").
- ****Sign of the effect**** — cheap to check against a stated expectation (attraction vs repulsion; bound vs unbound sign of E).

****Requires expert reasoning:****

- ****Whether the correct case-split was made**** (cavitation threshold 2025 A.2; stick/slip 2025 C.3, 2020 C.2; bound/unbound 2024 A-7). Detecting a ****missing**** regime is not a recompute — it needs modeling judgment.
- ****Frame/constraint correctness**** — verifying the centrifugal term belongs (2024 A-1), that the free-body diagram enumerated all forces (2025 C.1 lists tension/friction/weight), or that a constraint (inextensible cable, rigid rod, incompressible volume) was applied — semantic, not syntactic.
- ****Whether an approximation is used inside its regime**** (2023 B.5 small-slope; 2021 A.4 leading-order-in- k). A number can be "right" yet obtained illegitimately.
- ****Graph feature-correctness**** (slope signs, breakpoints, asymptotes, duty cycle) — 2025 B.2/C.3; partially checkable if the sketch is encoded as a function, but reading an image needs a vision/expert step.
- ****Physical-plausibility gating**** — 2022's rule "if an answer is physically unrealistic, no numeric credit" (e.g., T_1 should be $\sim$ Earth temperature) requires domain priors.
- ****Validity of an alternative derivation path**** — 2022/2021 accept multiple methods; deciding a ****novel**** path is sound is expert work.

5. Empirically-grounded candidate definition of "the reference solution is correct"

Grounded in what these six mechanics solutions actually contain, a reference solution to an IPhO-style mechanics problem is ****correct**** iff, for every sub-part, all of the following hold:

1. ****Final-answer correctness in the demanded form.**** The stated result matches type: symbolic answers are algebraically equivalent to a ground-truth expression ****and**** written only in the mandated symbol set (2025 A.1); numeric answers fall in the accepted tolerance band, carry correct units, and meet the sig-fig requirement (2022 " ≥ 2 s.f."; 2025 bands); thresholds state the correct inequality ****and**** its direction (2025 C.1/C.3; 2024 A-7/B-9); graphs reproduce every enumerated qualitative feature — slope signs, breakpoints, asymptotes, symmetry, limiting values (2025 B.2/C.3; 2024 A-2; 2021 B.2); scaling answers give the correct exponent (2022).
2. ****Derivation validity, not just the endpoint.**** Each result is obtained from correct governing physics (Newton's laws / hydrostatic balance / conservation of momentum, angular momentum, energy, volume) with no dimensionally-inconsistent step. This mirrors the marking reality that a right number from a wrong formula earns nothing (2022 Rule 1b) and a dimensionally wrong equation earns nothing even under follow-on.
3. ****Case completeness.**** Every physically reachable regime is identified and correctly bounded: contact conditions (stick/slip, static/kinetic — 2025 C, 2020 C), phase/cavitation thresholds (2025 A.2), bound/unbound (2024 A-7), and root-branch selection (2020 A.2, 2024 A-3). Silently collapsing to one case makes the solution ****under-specified**** even if that case's algebra is right.

4. **Internal consistency.** Symbols, sign conventions, chosen frame, figures, and chained results agree with one another: a downstream part must use the actual upstream result (2025 C.2→C.5; 2023 B.5→C.3), and any annotated figure must match the prose/algebra (2025's m_{add} shading rule).

5. **Approximation legitimacy.** Every truncation/limit is applied inside its regime and the retained order is the correct one (2021 A.4 leading-order-in- k ; 2023 B.5 small-slope; 2025 $S_{\text{tll } S_b, S_c}$).

6. **Non-uniqueness tolerance.** Correctness is path-independent: any derivation reaching (1) via valid physics counts, matching the official schemes that grade multiple methods as equivalent (2022, 2021 Method 1/2, 2024).

What is distinctively *mechanics* in this definition (vs physics generally), from the observed corpus:

- **Vector / frame / coordinate correctness** is first-class and frequently the crux: signed force directions (2025 A.1, C.2; 2021 A.1), rotating-frame + centrifugal potential (2024 A-1), fixed sign conventions for angles (2023).
- **Free-body / constraint reasoning** is the dominant derivation mode: enumerate all forces at equilibrium (2025 C.1), impose inextensible-cable / rigid-rod / incompressible-volume constraints (2025 C, 2020 C).
- **Case-splitting on contact conditions** (stick vs slip, static vs kinetic friction, rolling implicit) is pervasive and heavily weighted (2025 C.1/C.3, 2020 B.1/C.2) — a failure mode largely absent from, say, pure EM problems.
- **Conservation-law cross-checks** are the natural correctness oracle: energy/work (2025 A.1, C.4/C.5; 2023 A.1), angular momentum (2024 A-4), and volume/mass conservation of an incompressible fluid (2025 C.1–C.3) each provide an independent recomputation the verifier can exploit cheaply.

Verifier-design implication: for these mechanics problems, a cheap layer (dimensional check + numeric recompute against tolerance + limiting-case + symmetry + conservation-law cross-check + symbol-whitelist) can adjudicate a large share of correctness conditions, while an expensive expert layer is reserved specifically for **case-completeness, frame/constraint validity, approximation-regime legitimacy, and graph-feature correctness** — the four surfaces above that are semantic rather than syntactic.

Files read (all absolute):

Problems — `...\ipho\IPhO_2025_Q2.pdf`, `IPhO_2024_Q3.pdf`, `IPhO_2023_Q3.pdf`, `IPhO_2022_Q3.pdf`, `IPhO_2021_Q1.pdf`, `IPhO_2020_Q2.pdf` (all in full).

Solutions (skimmed for form) — `IPhO_2025_S2.pdf` (richest marking schemes), `IPhO_2024_S3.pdf`, `IPhO_2022_S3.pdf` (contains the T3 Scaling-laws scheme plus the explicit "General rules" on follow-on/dimensional errors), `IPhO_2021_S1.pdf`, `IPhO_2020_S2.pdf` (pho.rs HTML-style). I did not open `IPhO_2023_S3.pdf`; its solution form is nonetheless well-determined from the 2023 problem (one 2-s.f. numeric in A.1, otherwise symbolic "express-in-terms-of" and an ODE-to-closed-form solve), consistent with the other Japanese/European schemes.

<usage><subagent_tokens>256897</subagent_tokens><tool_uses>14</tool_uses><duration_ms>1731730</duration_ms></usage>

</task-notification>

Claude

Both input tracks are complete and rich. Now I'll spawn the adversarial synthesis subagent, feeding it both the first-principles and empirical outputs, tasked with debating the two classes of correctness definition and identifying simplifications for IPhO-mechanics vs general physics.

Claude

Let me persist both reports to files (durable record + clean input for the next subagent), then spawn the adversarial synthesis.

Claude

The adversarial synthesis subagent is running. I'll wait for it, then produce the combined final definition.

Claude

I'll wait for the adversarial synthesis subagent's report before combining everything into the final definition.

YOU

```
<task-notification>
<task-id>a6009d39b08e4b6ba</task-id>
<tool-use-id>toolu_017k28Z6Prv3WpF9ZfCyrfBh</tool-use-id>
<output-file>C:\Users\alens\AppData\Local\Temp\claude\C--Users-alens-iphO-problems-agentic-generator\bdd0a861-2623-4c72-b7e3-6c8688795fc2\tasks\la6009d39b08e4b6ba.output</output-file>
<status>completed</status>
<summary>Agent "Adversarial synthesis of two correctness classes" finished</summary>
<note>A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once.</note>
<result>I have both analyses. This is a reasoning-and-synthesis task, so I'll deliver the full debate and deliverables directly.
```

Adversarial Debate: Class G vs Class M Correctness for IPhO Mechanics

0. Precise definition of the two classes (steelmanned)

****Class G — General-physics maximal correctness.**** `S` is correct for `P` iff the six-part specification of Track 1 §8 holds: (I) well-posedness of `P`, (II) physical soundness, (III) derivational validity, (IV) faithfulness $P \rightarrow S$, (V) completeness (parts + cases + demanded form), (VI) certification-consistency as a derived cheap test. Class G assumes **nothing** about domain structure: it must adjudicate an EM boundary-value problem, a QM perturbation series, a stat-mech partition-function argument, and a GR geodesic calculation with the same machinery. Crucially, Class G cannot assume (a) that a cheap independent oracle exists, (b) that the case structure is finite/enumerable, (c) that the answer type is drawn from a small known set, or (d) that "physical soundness" is dominated by a handful of conservation laws. Every one of (I)–(V) is a first-class judgment that may, in the worst case, require full expert reasoning.

****Class M — IPhO-mechanics specialized correctness.**** Same six conditions as targets, but the *verification procedure* exploits four structural regularities empirically established in Track 2:

- ****(M1) Conservation oracles are cheap and re-derivable**:** energy/work, linear & angular momentum, incompressible volume/mass give an *independent* value for a quantity via a path disjoint from `S`'s.
- ****(M2) The state space is low-dimensional and countable**:** Newtonian/Lagrangian mechanics on finite DOF; determinacy reduces to DOF-vs-constraint counting plus dimensional determinacy.
- ****(M3) Case-splits are finite and enumerable from a fixed menu**:** stick/slip, static/kinetic onset, bound/unbound, contact/separation (cavitation), root-branch selection. The *taxonomy* of splits is known a priori.
- ****(M4) Answers cluster into ~6 pre-declarable types**:** symbolic-closed-form (+ symbol whitelist), numeric+units+tolerance, threshold/inequality+direction, graph-as-feature-list, scaling-exponent+mechanism, show-that/derive-to-given-form.

Class M trades away domain generality to get a mechanizable per-type checker plus a cheap conservation oracle. It keeps the full Class-G target but reroutes as much adjudication as possible from expensive expert reasoning to cheap automatable checks.

Axis 1 — Well-posedness of P

****Class-G proponent.**** Well-posedness must remain a first-class, separate verdict (Track 1 §8-I, "ill-posed" is a *distinct* output*). It cannot be folded into soundness: an `S` can be flawlessly reasoned yet solve an ill-posed `P` vacuously. In general physics, well-posedness is genuinely hard — existence/uniqueness for a PDE, gauge-fixing, boundary-condition sufficiency, whether a variational problem has a minimizer. There is no cheap universal determinacy test; it is inherently Tier-3.

****Class-M proponent.**** Well-posedness stays a separate verdict — I concede necessity — but mechanics makes the *common* failure modes cheaply screenable. Three cheap structural checks catch most ill-posedness:

1. ****Dimensional determinacy**:** can the requested quantity even be formed from the givens' dimensional basis? (Buckingham- π count.) Under-determination often shows up as "the answer would need a parameter not in `P`" — Track 1 §1.2 exactly.
2. ****DOF-vs-constraint counting**:** degrees of freedom minus holonomic constraints gives the dynamical dimension; if the

requested quantity depends on an unfixed initial condition or an unspecified constraint, it's under-posed. This is a finite integer count, near-free.

3. **Conserved-quantity budget**: number of independent conservation laws vs number of unknowns — an over/under-determination smell test.

These convert much of §1.1–1.2 (determinacy, non-triviality) into cheap arithmetic. What they do *not* catch: §1.3 consistency across modalities (figure vs prose vs numbers), §1.4 physical realizability, §1.6 semantic unambiguity of a frame/origin. Those stay expensive.

Adjudication. Well-posedness remains a *separate first-class verdict* in both classes — this is invariant and non-negotiable. But Class M earns a real, sound discount on the *determinacy/non-triviality* sub-conditions (§1.1–1.2, §1.5): dimensional-determinacy + DOF-vs-constraint counting are legitimate cheap *falsifiers* of well-posedness. Caveat, and it is sharp: these are *necessary, not sufficient* conditions for well-posedness. A problem can pass DOF-counting and dimensional determinacy and still be ill-posed via modality inconsistency (§1.3) or ambiguous frame/origin (§1.6) — and Track 2 shows figure↔prose contradiction (2025 m_add shading) and frame ambiguity (2024 rotating-frame) are *real, live* mechanics failure modes. So Class M's discount is: *cheap-screen the determinacy core, but retain an expert residue for cross-modal consistency and semantic/frame unambiguity.* Well-posedness does not collapse into soundness in either class.

Axis 2 — The reducible core

Class-G proponent. All six conditions are irreducible *as conditions* — you cannot delete faithfulness or completeness in any domain and still have a correctness definition. What varies is only cost, not membership. And in the general case, none of them is reliably cheap: soundness can hinge on a subtle regime argument, completeness on a non-obvious bifurcation, derivational validity on a step whose justification is a deep theorem.

Class-M proponent. Agreed that no condition is *deleted* — but membership-invariance is not the interesting claim; *cost-class migration* is. In mechanics:

- **Derivational validity (III)** partially automates: dimensional homogeneity per step, symbolic equivalence via CAS, numeric recompute-vs-tolerance are Tier-0/1 and, empirically (Track 2 §4), decisive for a large share (2024 B-3, 2021 A.5, all of 2022 dimensional cases).
- **Physical soundness (II)** partially automates via conservation cross-checks (Axis 3).
- **Completeness-of-cases (V-cases)** partially automates via the finite case menu (Axis 4).
- **Faithfulness (IV)** and **completeness-of-parts (V-parts)** do *not* cheapen — they are about the $P \rightarrow S$ relation, which is semantic and domain-independent.

Adjudication. The membership set {well-posedness, soundness, derivational validity, faithfulness, completeness} is *invariant across both classes* — Class M deletes nothing. The honest simplification is a *cost-class migration table*, not a shortening of the definition:

Class-G condition	Irreducible as a condition?	Migrates cheaper in M?	
--- --- ---			
Well-posedness (I)	Yes	Partly (determinacy core → cheap; consistency/frame → expensive)	
Physical soundness (II)	Yes	Partly (conservation-checkable slice → cheap; regime/law-choice → expensive)	
Derivational validity (III)	Yes	Largely (dim + CAS + numeric → Tier 0–1)	
Faithfulness (IV)	Yes	No — stays expensive	
Completeness-parts (V)	Yes	No (cheap to enumerate parts, but judging "answered" is semantic)	
Completeness-cases (V)	Yes	Partly (finite menu → structured check; novelty residue expensive)	

The reducible-*cost* core = {III largely, II partly, I partly, V-cases partly}. The irreducible-*cost* core (survives every specialization at expert cost) = *{faithfulness, the semantic residue of well-posedness, law-choice/regime soundness, case-completeness novelty, "answered-in-substance" judgment}*.

Axis 3 — Conserved quantities as a correctness oracle

Class-M proponent. This is Class M's crown jewel and it is *structurally* unavailable to general Class G. Mechanics almost always admits an *independent re-derivation* of a target quantity through a conservation law whose derivational path is *disjoint* from *S*'s chosen path. Energy methods vs force integration; angular-momentum conservation vs torque integration; incompressible-volume/mass bookkeeping. Track 2 §46 names these as "the natural cheap correctness oracle," with concrete hits (2025 volume/mass; 2024 A-4 $J=\mu a^2 \omega$). Because the oracle path is *independent*, agreement is not the vacuous self-consistency of §4 — it is a genuine second witness. This lets Class M *certify* a meaningful slice of physical soundness, not merely refute.

Class-G proponent. Beware the refute-vs-certify asymmetry (Track 1 §4, §7 doctrine: "cheap checks refute but only expensive reasoning confirms"). A passed conservation check certifies exactly one thing: that *S*'s final value is *consistent* with that one conservation law at the evaluated point*. It does *not* certify:

- that the *reasoning* produced the value (right-answer-wrong-reasoning, §6.1, survives a numeric conservation match perfectly);
- soundness of parts of *S* not constrained by that conservation law (the law is a scalar projection of a high-dimensional derivation);
- correctness away from the tested inputs (a coincidental algebraic match at one branch);
- and general physics often lacks a *cheap independent* oracle at all — the "independent" derivation may be as expensive as the original (e.g., recomputing a partition function two ways).

Adjudication. Class M has a *genuine and real* structural advantage here that Class G cannot assume in general — cheap, independent conservation re-derivation is a distinctive feature of finite-DOF mechanics. But the asymmetry from Track 1 is binding and must be stated precisely:

- **What conservation checks legitimately CERTIFY:** that *S*'s final expression/value lies on the constraint surface imposed by the conservation law(s) invoked. If *multiple independent* conservation laws (say energy AND angular momentum) all pin the answer and *S* matches all of them across the symbolic form (not just one numeric point), the certified slice grows and right-number-wrong-reasoning becomes progressively harder to sustain — a wrong derivation must now coincidentally satisfy several independent projections *as functions of the symbols*, which is strong (though still not total) evidence.
- **What they CANNOT certify:** the derivational provenance (§6.1), soundness of unconstrained sub-steps, or correctness of the *choice* to apply that conservation law (was momentum actually conserved here? an external impulse or non-conservative contact force silently breaks it — and *deciding whether the law applies* is itself a soundness judgment that can be expensive).

So the honest formulation: *conservation checks upgrade from pure falsifiers* (Track 1's default) toward partial certifiers in mechanics, but only for the projection they constrain, and only symbolically-across-the-form rather than numerically-at-a-point. They never discharge faithfulness or provenance. The strongest safe use: run conservation as a *high-precision falsifier first* (cheap reject), and treat multi-law symbolic agreement as a *soundness-confidence booster* that *reduces but does not eliminate* the expert-confirmation budget. A subtle trap Class M must encode: verifying that the conservation law is *legitimately applicable* (closed system, no external torque/impulse, genuine incompressibility) is a soundness precondition — using a violated conservation law as an oracle is a false certifier.

Axis 4 — Case-completeness

Class-G proponent. Case-completeness is one of the hardest expert judgments — detecting a *missing* bifurcation is detecting an absence, and absences are unbounded in general physics (a phase transition you didn't know to look for, a resonance, a symmetry-breaking branch). You cannot enumerate what you don't know exists. This is irreducibly Tier-3.

Class-M proponent. In mechanics the case-generating mechanisms are a *known finite menu*, and this is the single biggest expert→structured-check conversion available:

- contact conditions: stick/slip, static→kinetic onset;
- constraint activity: bound/unbound, contact/separation (normal force ≥ 0 , i.e. cavitation/lift-off);
- root selection: which branch of a quadratic/extremum (Track 2: 2020 A.2 $\pm\pi/4, \pm 3\pi/4$; 2024 A-3 middle Lagrange root);

- regime boundaries: small-angle vs not, leading vs subleading term.

Each has a *mechanical trigger* that can be checked: for every contact/constraint in the free-body diagram, test whether its defining inequality (friction cone $|f| \leq \mu N$; normal force $N \geq 0$; relative velocity sign) can change sign over the problem's domain. If it can and `S` didn't split there, flag a missed case. This converts "did the expert think of all cases?" into "for each enumerable constraint, did `S` check its activity boundary?" — a *structured checklist*, Track 2's "signature mechanics failure" (2025 A.2 cavitation, C.3 stick/slip; 2020 B.1, C.2) made systematic.

Adjudication. This is a *real and sound conversion* — the most valuable one Class M offers — but with a clearly bounded residue. The finite menu turns *most* case-completeness into a mechanizable check: enumerate every contact/constraint/root/regime-boundary in the free-body + constraint model, and for each, verify `S` either resolves the branch or justifies excluding it. That is genuinely cheaper than open-ended expert bifurcation-hunting and it is domain-warranted.

But the residue is exactly where Class M is unsafe if it over-trusts the menu:

- Menu-completeness is itself an assumption.** A problem can hide a case the standard taxonomy doesn't list (a parametric resonance, a geometric degeneracy, an internal collision, a constraint that becomes active only after a state change the checklist didn't anticipate). The checklist has high recall on *listed* case types and *zero* recall on unlisted ones.
- Building the constraint inventory correctly is itself the free-body/constraint-semantics judgment** (Axis 6) — if `S` (or the checker) mis-enumerates the forces/constraints, the case checklist inherits the error.

So: *case-completeness migrates from Tier-3 expert-judgment to a Tier-2 structured check over the enumerated constraint set*, with a retained Tier-3 residue for (a) confirming the constraint inventory is complete and (b) catching non-menu bifurcations. **Net:** a large discount, not a total one. Do not let the checklist's silence be read as certification.

Axis 5 — Answer-type coverage

Class-M proponent. Track 2 §1 empirically shows mechanics answers cluster into ~6 types. Because the type set is small and *declarable up front*, Class M can *pre-compile one checker per type*:

- symbolic → CAS equivalence + symbol-whitelist enforcement;
- numeric → tolerance-interval + units + sig-figs;
- threshold → inequality direction + boundary value + correct side;
- graph → enumerated-feature list (piece count, slope signs/order, breakpoints, asymptotes, symmetry, duty cycle) — *not* visual likeness;
- scaling → exponent + mechanism-gating;
- show-that → term-by-term match of the mandated intermediate objects.

Class G cannot assume the type distribution — a general verifier must first *classify* an answer of unknown type (a tensor field? a spectrum? a proof? a phase diagram?) before it can check it, and the long tail is unbounded.

Class-G proponent. Two cautions. First, the type is a property of the *demand* answer, and reading the demand off `P` correctly is a faithfulness question — mis-typing (treating a "show that" as a free symbolic, skipping the mandated intermediate match) silently drops a real requirement. Second, per-type checkers verify the *form and value* but not the *provenance*; a numeric-in-tolerance checker is blind to right-number-wrong-reasoning by construction.

Adjudication. Bounded, pre-declarable answer-type diversity is a *legitimate and clean Class-M advantage* — the per-type checker is sound *for the endpoint-form conditions* (Track 2 §2's "correct in demanded form"). It cleanly discharges the *form/value* slice of Completeness-(V) and the endpoint slice of final-answer correctness. Two boundaries: (1) the checker discharges *form and value only*, never provenance — it must be composed with derivational-validity checks, never used alone (Track 1 §6.1); (2) *selecting* the right per-type checker is a faithfulness read of `P`'s demand and must be verified, not assumed (mis-typing = silent requirement drop). With those two guards, this is the second-most-valuable Class-M simplification after the case menu, and it is safe.

Axis 6 — What does NOT simplify

Class-M proponent (honest limits). I concede these get *no* mechanics discount and Class M must invoke full Class-G machinery:

- Faithfulness $P \rightarrow S$ (IV).** This is the dominant generated-content failure mode (Track 1 §0, §7). It is a semantic relation between two artifacts — same system, data, geometry, BCs, all parts. Mechanics structure gives *zero* leverage: a generator can change a mass in `P` and not in `S`, drop a boundary condition, or produce a figure contradicting the prose in mechanics exactly as in EM. Every intrinsic check (soundness, derivation, even conservation) can pass on an `S` that faithfully solves the *wrong* problem.
- Frame/constraint semantic validity.** Whether the centrifugal term belongs, whether all forces are enumerated, whether a constraint is holonomic and correctly applied, whether the chosen frame is inertial — Track 2 §4 lists these as expensive, and they are *upstream* of every cheap check (the conservation oracle and case menu both depend on a correct free-body/constraint model). Mechanics makes this *more* central, not cheaper.
- Approximation-regime legitimacy.** Whether a truncation is applied inside its regime at the correct retained order (2021 A.4 keep $D^2 \propto k \square^2$ drop $p \cdot D$; 2023 B.5 small-slope with figures as deliberate traps). Deciding regime validity is a physical judgment; the deliberate-trap examples show it resists cheap checks.
- Genuinely novel derivations / alternative valid paths.** Non-uniqueness tolerance (Track 2 §6; 2022 multi-method, 2021 Method 1/2) means a correct `S` may follow a path the checker didn't anticipate; validating a novel path is expert work, and a conservation oracle can *corroborate the endpoint* but not *validate the novel argument*.

Class-G proponent. Exactly — and note these four are precisely the irreducible core from Axis 2. The specialization is real but *bounded by the semantic/upstream conditions*: everything cheap in Class M (conservation oracle, case menu, per-type checker) is *downstream of and conditional on* a correct frame/constraint model and a faithful reading of P — the two things that don't simplify. That is the structural reason the discount can never reach 100%.

Adjudication. Concurred, no dispute. The non-simplifying set = **{faithfulness $P \rightarrow S$; frame/constraint semantic validity; approximation-regime legitimacy; novel-derivation validation}**. This is invariant. Critically, note the **dependency ordering**: the cheap Class-M checks *presuppose* the expensive ones. The conservation oracle is only valid if the system is correctly identified as closed (a soundness/frame judgment); the case menu is only complete if the constraint inventory is correct (a frame/constraint judgment); the per-type checker is only correctly selected if `P`'s demand is faithfully read. So Class M is not "cheap layer OR expensive layer" — it is "cheap layer *gated on* a correct expensive-layer model." This is the key architectural truth for Axis 7.

Axis 7 — Cost / verifier architecture

Class-G cascade (Track 1 §7, domain-agnostic):

- Tier 0: dimensional homogeneity, units, sign/range, numeric re-eval from `S`'s own final expression, constants.
- Tier 1: symbolic CAS recompute of final steps, limiting cases, symmetry, order-of-magnitude.
- Tier 2: step-graph local-validity reconstruction; **faithfulness extraction**; completeness audit; independent re-derivation.
- Tier 3: well-posedness adjudication; regime validity; global soundness of subtle arguments; answer-vs-derivation disagreements.

Doctrine: cheap refutes, expensive confirms; spend expensive reasoning only where cheap is silent.

Class-M cascade differs in three ways:

- A conservation-oracle stage is promoted into Tier 1** (it is domain-guaranteed cheap and independent here, whereas Class G cannot count on it existing). Runs as a high-precision falsifier; multi-law symbolic agreement additionally feeds a soundness-confidence signal.
- A structured case-menu checklist becomes a Tier-2 mechanized pass** (enumerate constraints/contacts/roots/regimes → verify branch resolution), replacing open-ended expert bifurcation-hunting for the *listed* case types.
- Per-type answer checkers are pre-compiled and dispatched at Tier 0–1** by answer type read from `P`'s demand.

But — from Axis 6 — the Class-M cascade needs an **explicit upstream gate** the Class-G cascade folds into Tier 3: **build/verify the free-body + constraint + frame model, and read P's demand faithfully, FIRST**, because every downstream cheap stage is conditional on it. So the Class-M architecture is best drawn as:

GATE (expert, but small & structured): faithful read of P's demand

+ correct frame/constraint/free-body model + regime declaration

■ (if this is wrong, all cheap checks below are invalid)



Tier 0 dim homogeneity · units · sign/range · numeric re-eval · symbol-whitelist · answer-type dispatch

Tier 1 CAS symbolic-equiv · limiting/special cases · symmetry · CONSERVATION ORACLE (independent re-derivation)

Tier 2 step-graph local validity · CASE-MENU checklist · faithfulness extraction (parts coverage) · completeness

Tier 3 regime-legitimacy · novel-path validation · residual case-completeness (non-menu) · soundness of subtle steps

· adjudicate answer-vs-derivation conflicts

****Adjudication.**** The two cascades share Tier 0 almost identically. They diverge at Tier 1 (Class M *guarantees* a cheap independent oracle; Class G merely *hopes* for one) and Tier 2 (Class M's case-menu is structured; Class G's is open). The decisive architectural difference is the ****explicit GATE****: in Class G the frame/constraint/faithfulness judgment is diffusely spread across Tier 2–3; in Class M it should be *hoisted to the front as an explicit precondition*, because the entire cheap discount is conditional on it. Cost-optimally: spend a bounded expert budget up front to certify the model and the P-read, then let the cheap layer run wide, then spend residual expert budget only on regime/novelty/non-menu-cases.

DELIVERABLES

D1. Crisp statement of the simplification

Going from general physics (Class G) to IPhO mechanics (Class M), ****no correctness condition is deleted; several migrate from expensive to cheap, and a hard core stays expensive.**** Mapping each Class-G condition to its Class-M treatment:

Class-G condition (Track 1 §8)	Class-M treatment	Justification
---	---	---
I) Well-posedness — determinacy/non-triviality (§1.1–1.2, 1.5)	**Cheapen** to dimensional-determinacy +	
DOF-vs-constraint counting + conservation-budget (Tier 0–1 falsifiers)	Finite-DOF structure makes determinacy an integer	
count; necessary-not-sufficient screen		
I) Well-posedness — modality consistency + frame/semantic unambiguity (§1.3, 1.6)	**Unchanged** (expert)	
Figure↔prose and frame ambiguity are live mechanics failures; semantic, no structural shortcut		
II) Physical soundness — conservation/symmetry slice (§2.4)	**Cheapen** to conservation oracle (Tier 1), partial	
certifier for the constrained projection	Cheap independent re-derivation is a distinctive finite-DOF affordance	
II) Physical soundness — law choice, regime validity, applicability of conservation law (§2.1–2.3, 2.5)	**Unchanged**	
(expert)	Deciding *which* law applies / whether regime holds is judgment; deliberate-trap evidence	
III) Derivational validity (§3.1–3.5)	**Largely cheapen** — per-step dim homogeneity + CAS symbolic-equiv +	
numeric-vs-tolerance (Tier 0–1)	Empirically decisive across Track 2; mechanics steps are algebraic/CAS-tractable	
IV) Faithfulness $P \rightarrow S$	**Unchanged** (expert, Tier 2) — the dominant generated-content failure	
Semantic $P \leftrightarrow S$ relation; mechanics gives zero leverage		
V) Completeness — parts coverage & demanded form	**Cheapen** form/value via pre-compiled per-type	
checkers (Tier 0–1); parts-*enumeration* cheap, "answered-in-substance" unchanged	~6 declarable answer types; but	
judging substance stays semantic		
V) Completeness — case-completeness	**Cheapen** to Tier-2 case-menu checklist over enumerated	
contacts/constraints/roots/regimes; **residue unchanged** for non-menu bifurcations & inventory-completeness	Finite	
enumerable case taxonomy; but menu-completeness is itself assumed		
VI) Certification-consistency	**Cheapen/strengthen** — conservation + limiting + symmetry battery is a stronger	
high-precision test in mechanics	More independent cheap oracles available than general physics	

****One-line summary:**** Mechanics cheapens *derivational validity* (nearly fully), the *conservation slice of soundness*, the *determinacy core of well-posedness*, *case-completeness over the known menu*, and *answer-form completeness* — while leaving *faithfulness, frame/constraint semantics, regime legitimacy, law-applicability, non-menu case-hunting, and novel-path validation* at full Class-G expert cost. And every cheap check is ****conditional on a correct frame/constraint**

model and a faithful read of P^* , which do not cheapen.

D2. The irreducible core (survives every specialization)

No specialization removes these; they must be adjudicated at expert cost in every domain including mechanics:

1. **Faithfulness $P \rightarrow S^*$** — that S^* solves the very problem P poses (same system, data, geometry, BCs, all parts). **The* generated-content failure mode.**
2. **Frame/constraint/free-body semantic validity** — correct force enumeration, constraint identification and application, inertial-vs-non-inertial frame, legitimacy of pseudo-forces. (Upstream of every cheap check.)
3. **Law-choice and approximation-regime legitimacy** — which laws apply, whether a conservation law is actually valid here, whether a truncation sits inside its regime at correct order.
4. **Non-menu case-completeness and constraint-inventory completeness** — catching bifurcations outside the standard taxonomy, and certifying the taxonomy was fully instantiated.
5. **Validation of genuinely novel / alternative derivation paths.**
6. **Provenance** — that the right value arises **because of** the reasoning (right-answer-wrong-reasoning is incorrect for a reference solution); no endpoint check certifies this.

D3. Recommended two-layer definition of correctness for IPhO mechanics

A reference solution S^* to an IPhO-style mechanics problem P is **correct** iff it passes the operational core AND the expert residue, evaluated per sub-part. The layers are ordered: **the expert gate model is a precondition for trusting the cheap core.**

Layer 0 — Expert GATE (small, structured, must run first):

- Faithful extraction of P 's demand per sub-part (system, data, geometry, BCs, requested quantity + demanded answer type).
- Correct free-body / constraint / frame model; declaration of the intended regime and idealizations.
- **This layer is cheap-ish to state but expert to certify; if it is wrong, Layer 1 results are void.**

Layer 1 — Cheap-checkable operational core (mechanizable, exploits mechanics structure):

1. **Well-posedness screen:** dimensional determinacy + DOF-vs-constraint count + conservation-budget (falsify under/over-determination).
2. **Derivational validity:** per-step dimensional homogeneity; CAS symbolic-equivalence of final and key intermediate expressions; numeric recompute within stated tolerance/units/sig-figs.
3. **Conservation oracle:** independent re-derivation of the target via energy / momentum / angular-momentum / volume-mass; require symbolic (not just point-numeric) agreement across all applicable independent laws. Falsifier first; multi-law agreement is a soundness-confidence booster.
4. **Consistency battery:** limiting/special cases, symmetry, sign/direction, order-of-magnitude, cross-part chaining (downstream uses actual upstream result).
5. **Answer-form check (per-type, pre-compiled):** symbolic-equiv + symbol-whitelist / numeric-tolerance+units+sigfigs / threshold inequality+direction+side / graph enumerated-feature list / scaling exponent+mechanism / show-that term-by-term.
6. **Case-menu checklist:** for every enumerated contact/constraint/root-branch/regime-boundary, verify S^* resolves the branch or justifies exclusion.

Layer 2 — Expert-required residue (expensive, only where Layer 1 is silent or Layer 0 flags risk):

- Faithfulness confirmation and "answered-in-substance" for each part.
- Frame/constraint/law-applicability soundness (including: is each invoked conservation law actually valid here?).
- Approximation-regime legitimacy and retained-order correctness.
- Residual case-completeness: non-menu bifurcations, constraint-inventory completeness.
- Novel/alternative-path validation.
- Provenance adjudication and any answer-vs-derivation conflict.

Correctness verdict = Layer-0 gate passes $\wedge$ Layer-1 core passes $\wedge$ Layer-2 residue passes, per sub-part, path-independently (non-uniqueness tolerance). Well-posedness failure yields the distinct "ill-posed" output, not "incorrect S^* ."

D4. Explicit caveats — where the mechanics simplification is UNSAFE

1. **Right-number-wrong-reasoning survives the cheap core.** Conservation oracle + numeric-tolerance + answer-form checks are all **endpoint** checks; a flawed derivation that coincidentally lands on the constraint surface passes them. Never certify on Layer 1 alone — provenance is Layer 2. (Track 1 §6.1.)
2. **Conservation oracle as a false certifier.** If the conservation law is silently inapplicable (open system, external impulse/torque, non-conservative contact, only-approximate incompressibility), a "passing" oracle certifies nothing and may actively mislead. Applicability is a Layer-2 soundness precondition, not a free check.
3. **Missed non-menu case.** The case-menu checklist has zero recall on bifurcations outside the standard taxonomy (parametric resonance, geometric degeneracy, post-state-change constraint activation). Checklist silence $\neq$ case-completeness.
4. **Corrupted constraint inventory poisons everything downstream.** If Layer 0's free-body/constraint model is wrong, the conservation oracle, case menu, and DOF count all inherit the error while still "passing." The gate is load-bearing.
5. **Unfaithful S passing all intrinsic checks.** An *`S`* that flawlessly and soundly solves a **different** problem than *`P`* (changed datum, dropped BC, figure-prose mismatch) passes soundness, derivation, and conservation entirely. Faithfulness is orthogonal to the cheap core and must be independently established. This is the dominant generated-content risk and receives **no** mechanics discount.
6. **Answer mis-typing silently drops requirements.** Selecting the wrong per-type checker (e.g., treating a mandated "show-that" as free symbolic, or missing a symbol-whitelist restriction) passes a weaker test than demanded. Type selection is a faithfulness read, not an assumption.
7. **Point-numeric conservation agreement over-trusted.** Agreement at a single evaluated input can be coincidental; require symbolic-across-the-form agreement before treating the oracle as a soundness booster rather than a mere spot falsifier.

Bottom line: The mechanics specialization is a **cost migration**, not a **definition reduction**. It safely and substantially cheapens derivational validity, the conservation slice of soundness, the determinacy core of well-posedness, answer-form completeness, and menu-case-completeness — provided the verifier (a) treats all cheap checks as falsifiers-plus-bounded-certifiers gated on a correct Layer-0 model, and (b) reserves an irreducible expert budget for faithfulness, frame/constraint/law-applicability, regime legitimacy, non-menu cases, novel paths, and provenance. Any architecture that lets the cheap layer **certify** correctness on its own is unsound.

<usage><subagent_tokens>37980</subagent_tokens><tool_uses>2</tool_uses><duration_ms>193343</duration_ms></usage>
</task-notification>

Claude

All three tracks are complete. Now I'll combine them into the final definition of correctness. Let me update tracking and check the target directory.

Claude

There's a prior *`DEFINITION.MD`* (deleted in the working tree). Let me read the rest to align with existing framing, then write the superseding combined version.

Claude

The prior definition is strong but — being framed around grading-style step credit — it under-weights the single most important point for **generated** content: **faithfulness** (does S solve the P that was actually rendered?). My fresh three-track synthesis foregrounds that, adds well-posedness as a first-class "ill-posed" verdict, and adds the Layer-0 gate. I'll write the combined, superseding definition to the tracked path.

Claude

Let me update the project memory to record the completed definition, then summarize.

Claude

Done. Here's what I produced and where it lives.

What I did

****Pulled the real problems**** (via `curl` — PowerShell's downloader silently failed in this sandbox). The 5 most-recent genuine mechanics theory problems from the archive, plus the purest anchor:

Year	Q	Problem	Mechanics content
2025	Q2	Cox's Timepiece	hydrostatics, buoyancy, dry friction, PV energy cycle
2024	Q3	Black Widow Pulsar	orbital dynamics, Lagrange points, stellar hydrostatic stability
2023	Q3	Water and Objects	energy conservation, capillary statics, floating-body balance
2022	Q3	Scaling Laws	beam/capillary statics, damped hydrodynamics (Part C is SR — flagged)
2021	Q1	Planetary Physics	isostasy/hydrostatics (Part B is seismic-ray — flagged)
2020	Q2	Anisotropic Friction	*purest anchor* — friction EOMs, curvature, rigid-rod, stick/slip

****Ran the three tracks you asked for****, in the dependency order you specified:

- ****Track 1 (parallel subagent, first-principles, blind)**** — a 6-condition spec (well-posedness, soundness, derivational validity, faithfulness, completeness) with a cost-tiered cascade.
- ****Track 2 (empirical, from the actual PDFs)**** — cataloged ~6 recurring answer types, mechanics-specific failure surfaces (sign/direction, contact-condition case-splits, root branches, frame errors), and which checks are cheap vs expert.
- ****Track 3 (adversarial subagent, Class G vs Class M)**** — debated general-physics vs mechanics correctness and produced the key result: ****mechanics is a cost *migration*, not a definition *reduction*.**

The combined definition

Written to ****verifier_experiments/DEFINITION.MD**** (supersedes the prior deleted version). The headline conclusions:

1. ****Correctness of the reference solution $\neq$ grading.**** It's three separate verdicts: ****well-posedness of `P`**** (emit a distinct ****ILL-POSED****, not "S wrong"), ****intrinsic soundness of `S`****, and ****faithfulness `P $\rightarrow$ S`****.
2. ****Faithfulness is the dominant generated-content failure**** (S elegantly solves a **drifted** version of P) and gets ****zero discount**** from mechanics — no intrinsic check catches it.
3. ****Right-answer-wrong-reasoning = INCORRECT**** for a reference solution; provenance is never certified by endpoint checks.
4. ****Mechanics cheapens**** derivational validity (dims + CAS + numeric), the conservation slice of soundness (independent re-derivation oracle), the determinacy core of well-posedness (DOF-vs-constraint counting), answer-form completeness (per-type checkers), and case-completeness (finite stick/slip / bound-unbound menu) — but leaves an ****irreducible expert core****: faithfulness, frame/constraint semantics, regime legitimacy, non-menu cases, novel-path validation.
5. Operationalized as a ****Layer-0 gate $\rightarrow$ cheap Layer-1 cascade $\rightarrow$ escalation-only Layer-2****, with 7 explicit "where this is unsafe" caveats.

Intermediate artifacts are in the scratchpad (`track1\_first\_principles.md`, `track2\_empirical.md`) and the PDFs under `scratchpad/iphO/`. Memory updated.

One flag worth your call: ****2022 (SR leak) and 2021 (seismic-ray leak) are the two least-pure mechanics anchors.**** If you'd rather the anchor set be maximally mechanics-pure than strictly most-recent, I'd swap those for 2020 Anisotropic Friction and one more. Want me to lock the anchor set, or draft the structured-verdict output schema next?

YOU

```
<task-notification>
<task-id>bv2wk2a1b</task-id>
<tool-use-id>toolu_018tw5mYTewwenujELPQS1UE</tool-use-id>
<output-file>C:\Users\alens\AppData\Local\Temp\claude\C--Users-alens-iphO-problems-agentic-generator\bdd0a861-2623-4c72-b7e3-6c8688795fc2\tasks\bv2wk2a1b.output</output-file>
<status>completed</status>
<summary>Background command "Download candidate IPhO problem and solution PDFs" completed (exit code 0)</summary>
</task-notification>
```

Claude

That's just the original PowerShell download job finally exiting — no action needed; I already completed everything via `curl` and delivered the definition. Nothing further to do unless you'd like me to lock the anchor set or draft the structured-verdict schema.

YOU

```
<task-notification>
<task-id>bna49jvuz</task-id>
<task-id>bwx0h0ktm</task-id>
<task-id>__orphan_summary__:shell</task-id>
<status>stopped</status>
<summary>2 background shell command task(s) from the previous session have no completion record. They may have
been stopped (via the UI, Monitor timeout, or agent teardown — these leave no transcript marker), or they may have been
running when the previous Claude Code process exited. They have been marked stopped. Task ids: bna49jvuz,
bwx0h0ktm. Task ids in this notification beginning with "__orphan_summary" are internal scan markers, not
tasks.</summary>
</task-notification>
```

YOU

I want now to start writing report for the assignment. I want to say how with every optimization, it's crucial to first understand the business context. Who is paying for the model costs in the first place so we understand stakeholders.

Frame that my approach to the assignment is to frame as exploration and experimentation prior to talking to domain experts who are actually doing the grading, so that I can think about 1) cost optimization in real time, when they suggest a certain step is really important and should be custom LLM call 2) know how certain cost optimizations affect quality

As assingment reviewer is not a tech or physics domain expert, I like storytelling a little in simple terms so that one thing is communicated at a time in an actual real world context.

Give me at most 8 sentences as intro and start like below, framing clearly how we are approaching this assignment as I told you above:

Isaac is a physics professor who will be creating problems w/ solutions for this years International Physics Olympiad, that is a competition etc etc...

Claude

Isaac is a physics professor who each year helps write the problems and worked solutions for the International Physics Olympiad, the world championship where the brightest high-school physicists compete. Writing these is slow and expensive, so his team is building an AI pipeline that drafts a problem and its solution automatically — but before any solution can be trusted, something has to check that the solution is actually **correct**. That checking is done by asking a powerful AI model, and every one of those checks costs money; the person paying that bill is not Isaac but the program's budget owner, who cares about getting reliable checks without the costs quietly ballooning as they generate thousands of problems. So the real question of this assignment is not just "can we verify a solution?" but "can we verify it **cheaply enough** that the whole pipeline stays affordable at scale?"

My approach is deliberately to treat this first stage as exploration and experimentation, not a finished answer, because the people who truly know what "correct" means here are the domain experts who grade these problems by hand — and I have not spoken with them yet. By building and testing early, I put myself in a position to sit down with a grader and react in real time: when they tell me a particular step is the crux of the problem and must never be gotten wrong, I can decide on the spot to spend more and give it its own dedicated AI check. Just as importantly, having already measured how each cost-saving shortcut affects accuracy, I can tell them honestly what quality we would trade away to save money, instead of guessing. In short, this report is me learning the terrain first, so that the expensive conversation with the experts later is fast, concrete, and grounded in numbers rather than opinions.

YOU

first sentence should make it clear why I am storytelling

the sentences should be shorter

Claude

To keep this grounded in the real world rather than abstract jargon, let me tell it as a short story. Isaac is a physics professor. Every year he helps write the problems and worked solutions for the International Physics Olympiad, the world championship for high-school physicists. Writing them by hand is slow and costly, so his team is building an AI pipeline to draft each problem and its solution. But no solution can be trusted until something checks that it is actually correct. That check is another AI call, and every call costs money. The person paying is not Isaac — it is the program's budget owner, who wants reliable checks without costs ballooning across thousands of problems. So the real question is not just "can we verify a solution?" but "can we verify it cheaply enough to run at scale?"

Then a second short paragraph for the approach:

My plan is to treat this stage as exploration, not a final answer. The people who truly know what "correct" means are the experts who grade these problems by hand, and I have not spoken with them yet. So I build and test early to prepare for that conversation. When a grader says one step is critical, I can decide on the spot to give it its own dedicated AI check. And because I will have already measured how each shortcut affects accuracy, I can tell them honestly what quality we trade to save money. This report is me learning the terrain first, so the expensive expert conversation later is fast and grounded in numbers.

YOU

My plan is to treat this stage as exploration, not a final answer. -> lets make this more clear w/ more info, like this -> I am framing your assignment as doing an exploration phase before talking to Isaac, so that when he...

Claude

I am framing this assignment as an exploration phase before sitting down with Isaac, so that when he tells me how grading really works, I am ready to react. Isaac and his fellow graders are the people who truly know what "correct" means, and I have not spoken with them yet. So I build and test early to prepare for that conversation. Then when a grader says one step is critical, I can decide on the spot to give it its own dedicated AI check. And because I will have already measured how each shortcut affects accuracy, I can tell him honestly what quality we trade to save money. This report is me learning the terrain first, so the expensive expert conversation later is fast and grounded in numbers.

YOU

To keep this grounded in the real world rather than abstract jargon, let me tell it as a short story. -> and add here smth to the effect that I am framing this assignment in a specific business context as it's safe to switch into engineering mode only when you understand the business context - who is paying for the model use, who are all the stakeholders, what are the pain points

Claude

To keep this grounded in the real world rather than abstract jargon, let me tell it as a short story. I am deliberately framing the assignment inside a specific business context first, because it is only safe to switch into engineering mode once you understand that context — who is paying for the model usage, who all the stakeholders are, and where the real pain points sit. Optimizing before you know those things means optimizing the wrong thing. So the story comes first, and the engineering follows from it. Isaac is a physics professor. Every year he helps write the problems and worked solutions for the International Physics Olympiad, the world championship for high-school physicists. Writing them by hand is slow and costly, so his team is building an AI pipeline to draft each problem and its solution. But no solution can be trusted until something checks that it is actually correct. That check is another AI call, and every call costs money. The person paying is not Isaac — it is the program's budget owner, who wants reliable checks without costs ballooning across thousands of problems. So the real question is not just "can we verify a solution?" but "can we verify it cheaply enough to run at scale?"

YOU

here's my more succinct and human readable version, but I want to make last paragraphs more about the fact that i'll be able to discuss cost, and cost vs quality, and currently last paragraphs read more "quality", then "cost"

I am framing the assignment inside a specific business context, because it is only safe to switch into engineering mode once you understand that context — who is paying for the model usage, who all the stakeholders are, and where the real pain points sit.

For the sake of efficient communication, let me frame it as a story.

Isaac is a physics professor. Every year he helps write the problems and worked solutions for the International Physics Olympiad, the world championship for high-school physicists.

Writing them by hand is slow and costly, so his team is building an AI pipeline to draft each problem and its solution. But no solution can be trusted until something checks that it is actually correct. That check is another AI system, and every call costs money.

I am framing this assignment as an experimentation phase before sitting down with Isaac, so that when he tells me how correctness works, I can have efficient conversations. Isaac and his fellow graders are the people who truly know what "correct" means, and I have not spoken with them yet. So I build and test early to prepare for that conversation.

Then when a grader says one step is critical, I can decide on the spot to give it its own dedicated AI check. And because I will have already measured how each shortcut affects accuracy, I can tell him honestly what quality we trade to save money. This report is me learning the terrain first, so the expensive expert conversation later is fast and grounded in numbers.

Claude

Here are the last two paragraphs reworked so cost — and the cost-versus-quality trade-off — leads, with quality as the thing we measure against it:

I am framing this assignment as an experimentation phase before sitting down with Isaac, so that when we talk, I can discuss cost in concrete terms. My goal going in is to already know the levers: which shortcuts make verification cheaper, and how much accuracy each one costs. Isaac and his fellow graders are the people who truly know what "correct" means, and I have not spoken with them yet. So I build and test early to put a price and a quality number on each option before that conversation.

Then when a grader tells me one step is critical, I can weigh it on the spot — spend more and give that step its own dedicated AI check, or save money where accuracy matters less. Because I will have already measured the cost and the quality of each shortcut, the trade-off becomes a real decision we make together, not a guess. This report is me learning the terrain first, so the expensive expert conversation later is fast, and grounded in numbers on both cost and quality.

Chat 3 — Assignment framing

Source: /chats/3.md

YOU

I am building an AI pipeline for building international physics olympiad level physics problem sets used for preparing students for competitions. When we build a problem text (PDF) and a solution, we have to evaluate if it is correct, and we want to build cost-optimized verifier that suits that kind of problems.

Let's think in context of the following assignment I got:

****Assignment****

Design an AI execution system capable of reducing inference costs by 50% while maintaining equal quality.

You may interpret, scope, and approach the problem as you consider appropriate. You may focus on a particular workload, application, model family, infrastructure layer, or combination of techniques.

We are interested not only in your proposed solution, but in how you define the problem, test your assumptions, use AI tools, make engineering tradeoffs, and determine what can credibly be demonstrated within the available time.

Your submission may include working software, experiments, simulations, benchmarks, architectural designs, technical analysis, or another form of evidence that you consider appropriate.

****Required submission****

Provide:

1. Your solution in any form you wish
2. All of your interactions with all AI models including all prompts and responses related to this assignment pasted into one text document (Word or alternative).

For the AI-use record, it is welcome, but not necessary, if you distinguish among:

- Prompts produced primarily by AI;
- Prompts developed jointly through AI interaction and your own reasoning;
- Prompts created independently by you.

There is no prescribed architecture, technology stack, model provider, or expected final form. We will evaluate the quality of your reasoning, technical judgment, execution, evidence, originality, and use of AI.

I want you to think about the definition of words used in the assignment as it relates to my proposed problem that I'd like to work on for the assignment. For ex. even though they used word inference, from the rest of the assignment text as they're using words like workload and application, it makes sense that we can understand inference in it's widest meaning -> AI pipeline/AI system inference. Think through rest of the terms so we make sure that my idea aligns.

Claude

Good instinct on "inference" — and the same move works for almost every other term in that brief. The assignment is written in deliberately system-level vocabulary, which means it's inviting you to optimize a *pipeline*, not a kernel. Here's how each load-bearing word maps onto your verifier, and which ones you actually need to nail down before the framing is safe.

****"AI execution system."**** Not a model — a *runtime* that orchestrates model calls, tools, and control flow to do work. This is the most permissive term in the brief. Your verifier is exactly this: it takes a problem PDF + candidate solution and runs a sequence of checks (parsing, dimensional analysis, numeric/symbolic verification, LLM adjudication) to emit a correctness judgment. "Execution" also implies inference-time, not training — so you're never on the hook for fine-tuning, which is the right scope for the time budget.

inference costs. You've got this right — read it as the total cost of running the system to produce one output, i.e. **\$ per problem-set verified** (tokens × price, summed across every model/tool call in the pipeline, not one forward pass). Making this interpretation **explicit** in the writeup is worth doing, because "how you define the problem" is something they say they're grading.

workload. A bounded class of tasks the system handles. Yours is crisp: **verifying IPhO-mechanics-level problems and solutions for correctness.** The brief explicitly blesses focusing on "a particular workload," and verification is a genuinely good one to pick because its economics are asymmetric — checking is often far cheaper than generating, and many errors (dimensional, boundary-case, sign, order-of-magnitude) are cheaply detectable without a frontier model. That asymmetry is **where your 50% actually comes from**, so name it early.

application. The end-use context — competition-prep problem generation. The verifier is one component of that; you're optimizing the component, not the whole app. Fine, but say so, so the boundary is clear.

"model family" / "infrastructure layer" / "combination of techniques." These three are the brief telling you which levers are on the table: routing across model sizes (cheap model for easy checks, frontier only when needed), serving-layer tricks (prompt caching, batching), and — crucially — **combining** them. A cascade + tool-use (symbolic solver / code execution) + prompt caching verifier is squarely inside what they're asking for.

Now the two terms that are load-bearing and where framings quietly go wrong:

reducing by 50%. A percentage is meaningless without a baseline, and the credibility of your whole submission rests on that baseline being **fair** — what a competent engineer would actually ship (e.g. "frontier model reads problem + solution with a strong verification prompt, every time"), not a strawman you sandbagged so your number looks good. Evaluators reading these look hard for exactly this. Define the baseline first, measure its cost and quality, **then** show your optimized system halves the cost.

maintaining equal quality. This is a **non-inferiority** constraint, not an improvement target — and it forces you to define what "quality" means for a verifier, which is your best "test your assumptions" moment. Raw accuracy is the wrong metric here: a verifier that passes a wrong solution (false negative) is far more damaging to the application than one that flags a correct solution for human review (false positive). So your quality metric should probably be **recall on catching genuine errors** (or a cost-weighted error rate), held equal to baseline, while cost drops. Calling that out shows judgment the accuracy-number crowd will miss.

Put together, an aligned one-line problem statement you could drop into the writeup:

> **Given the workload of verifying IPhO-level physics solutions, design an inference-time execution system that halves cost-per-verification versus a strong single-frontier-model baseline, while holding error-catching recall statistically non-inferior — via a cascade that spends cheap deterministic and small-model checks first and escalates to a frontier model only on genuinely ambiguous cases.**

That fits every term in the brief without stretching any of them.

One thing worth deciding before you build: do you scope strictly to **the verifier's own cost** (cleanest, most demonstrable in limited time), or do you go broader and argue the verifier **enables** cheaper generation (generate-with-cheap-model-then-verify beats one expensive generation)? The second is more ambitious but much harder to hold quality constant and to demonstrate credibly in the time. Which of those two are you leaning toward — I can pressure-test the baseline and eval design for whichever you pick.

YOU

"while holding error-catching recall statistically non-inferior" -> what do you mean by this, make simpler

"via a cascade that spends cheap deterministic and small-model checks first and escalates to a frontier model only on genuinely ambiguous cases." -> here you're pointing to a solution but the general assignment is technically more open ended, so it's prob more robust problem statement if we don't specifically say into which technical direction to go

Claude

On the first phrase — "statistically non-inferior recall" just means: **the cheap system catches wrong solutions at least as reliably as the expensive baseline does.** "Recall" = of all the genuinely broken solutions, what fraction did the verifier flag. "Non-inferior" = not meaningfully worse — and "statistically" is only there to say a tiny gap that's within measurement noise doesn't count as worse. Plain version: **while catching bad solutions just as reliably as the baseline.**

On the second point — you're right, and it's the more important catch. A problem statement that names the cascade has already smuggled the answer in, which weakens exactly the thing they're grading ("how you define the problem" as distinct from how you solve it). It also boxes you in: if mid-build the evidence says routing-by-difficulty beats a cascade, or that a symbolic-solver-first design wins, your own problem statement is now fighting you. Keep the statement about *what must be true of the system*; let the solution section own *how*.

Stripped-down version:

> *Given the workload of verifying IPhO-level physics solutions, design an inference-time execution system that cuts cost-per-verification by 50% versus a strong single-frontier-model baseline, while catching bad solutions at least as reliably as that baseline.*

Four things pinned, zero mechanism: the workload, the baseline to beat, the cost target, the quality floor. Everything about *how* — cascades, tool use, routing, caching, cheaper models, or some combination — stays in the solution space where you can let evidence pick the winner.

If you want to leave yourself even more room, you can generalize "single-frontier-model baseline" to "a strong baseline that a competent engineer would ship" — but I'd keep it concrete, because a vague baseline is the easiest place for a 50%-reduction claim to lose credibility. Naming it fixes the reference point you're measuring against.

YOU

i am writing intro for the report for the assignment and it goes like this, I want you to in two paragraphs talk about why this problem statements fits the assignment (the chatting on terms we did above) and actually put in the latest problem statement we did above

> I am framing the assignment inside a specific business context, because it is only safe to switch into engineering mode once you understand that context — who is paying for the model usage, who all the stakeholders are, and where the real pain points sit.

>

> For the sake of efficient communication, let me frame it as a story.

>

> Isaac is a physics professor. Every year he helps write the problems and worked solutions for the International Physics Olympiad, the world championship for high-school physicists.

>

> Writing them by hand is slow and costly, so his team is building an AI pipeline to draft each problem and its solution. But no solution can be trusted until something checks that it is actually correct. That check is another AI system, and every call costs money.

>

> I am framing this assignment as an experimentation phase before sitting down with Isaac, so that when we talk, I can discuss cost in concrete terms.

>

> My goal going in is to already know the levers: which shortcuts make verification cheaper, and how much accuracy each one costs

>

> Then when a grader tells me one step is critical, I can weigh it on the spot — spend more and give that step its own dedicated AI check, or save money where accuracy matters less.

>

> Because I will have already measured the cost and the quality of some of the shortcuts.

>

> Let's get to precisely defining what we're solving for.

Claude

Here's a version that picks up right where "Let's get to precisely defining what we're solving for" leaves off:

Concretely, the problem I'm solving is this:

> *Given the workload of verifying IPhO-level physics solutions, design an inference-time execution system that cuts cost-per-verification by 50% versus a strong single-frontier-model baseline, while catching bad solutions at least as reliably as that baseline.*

This is a faithful reading of the assignment, not a narrowing of it. The brief asks for an "AI execution system" that reduces "inference costs" — and its own vocabulary (workload, application, model family, infrastructure layer, combination of techniques) signals that "inference" is meant broadly: the cost of running a pipeline to produce an output, not one model's forward pass. So I read inference cost as dollars per solution verified, summed across every model and tool call the system makes. Verification is a deliberately chosen workload rather than a generic one, because its economics are asymmetric — checking a solution is often far cheaper than producing it, and many errors are detectable without the most expensive model. That asymmetry is where a real cost reduction can come from, which is exactly why it makes a good target for a 50% claim.

The statement also pins down the two things a percentage claim needs to be credible, and nothing more. It fixes a reference point — a strong single-frontier-model baseline, the kind a competent engineer would actually ship, not a strawman inflated to make the number look easy — and it treats "equal quality" as a floor rather than a goal: the cheap system has to catch bad solutions at least as reliably as that baseline, so any savings I report are savings at held-constant quality, not quality traded away for cost. What it deliberately leaves open is *how*. It names no cascade, no router, no caching scheme, because the assignment is open-ended on technique and the honest version of this work lets the evidence decide which levers pay off. That keeps the definition of the problem separable from the solution to it — which, given the story above, is the whole point: I want to walk into the conversation with Isaac already knowing what each shortcut costs and what it buys.

YOU

export this entire chat in md file I can download where its clear what your msgs are, and what are mine

Claude

(This file.)

Chat 4 — IPHO solution PDFs with test errors

Source: /chats/4.zip · model claude-opus-4-8

YOU

there are i believe 5 ipho solution pdfs (if not download pdfs), as we need for testing our system 2 pdf solutions w/ errors in them - take 10 different types of errors (based on our definition of correctness in definition md) and edit two pdf solutions so there are actual 10 errors (different types!)

there are i believe 5 ipho solution pdfs (if not download pdfs), as we need for testing our system 2 pdf solutions w/ errors in them - take 10 different types of errors (based on our definition of correctness in definition md) and edit two pdf solutions so there are actual 10 errors (different types!), and document in md file what you did

Claude

I'll start by looking at the repo state and the definition doc.

YOU

Continue from where you left off.

Claude

No response requested.

YOU

there are i believe 5 ipho solution pdfs (if not download pdfs), as we need for testing our system 2 pdf solutions w/ errors in them - take 10 different types of errors (based on our definition of correctness in definition md) and edit two pdf solutions so there are actual 10 errors (different types!), and document in md file

Claude

Found the right URL path (`pdf/...`) and a real Python install.

Claude

Bash cwd persisted; using absolute paths. Installing PDF tooling in parallel.

Claude

All 12 PDFs downloaded. `pikpdf` dragged in `lxml`, which won't build on Python 3.15 beta — installing `pymupdf` alone.

Claude

pymupdf works. Now let me read the two purest mechanics anchors — 2020 S2 (Anisotropic Friction) and 2023 S3 (Water and Objects).

Claude

2023 S3 and 2021 S1 are clean official LaTeX PDFs with excellent text extraction — the best candidates. Let me read 2021 S1 in full plus both problem statements (needed to design faithfulness errors).

Claude

Now the problem statements, which I need to design faithfulness errors and check symbol whitelists.

Claude

2021 Q1 gives me explicit symbol whitelists (A.3, A.4) — useful. Now 2023 Q3.

Claude

I have everything I need. Let me finalize the design and probe the PDF internals for how to edit them.

****The 10 error types**** (spanning the DEFINITION.MD conditions — note (I) well-posedness is excluded by construction, since it's a property of `P`, and we're editing only `S`):

#	Doc	Part	Error type	Condition	Cheapest catching check
1	2023 S3	A.1	Dimensionless geometric factor	III	Tier 2 (dim-blind)
2	2023 S3	B.3	Sign/direction	III	Tier 0.3
3	2023 S3	B.4	Dimensional inconsistency	III	Tier 0.1
4	2023 S3	B.5	Wrong BC / root branch	III	Tier 1.2
5	2023 S3	C.2	Symbol-whitelist violation	V	Tier 0.5
6	2021 S1	A.1	Faithfulness drift $P \rightarrow S$	IV	Tier 2.5
7	2021 S1	A.4	Wrong retained order	II	Tier 2.4
8	2021 S1	A.5	Right-answer-wrong-reasoning	III	Tier 2.1
9	2021 S1	B.2	Conservation/energy-budget violation	II	Tier 1.3
10	2021 S1	B.3	Completeness — answer not delivered	V	checklist

Claude

2023 S3 uses Latin Modern Math in Identity■H CID subsets, so byte-level stream swaps aren't practical. I'll use redact-and-reinsert with Cambria Math. Let me verify glyph coverage first.

Claude

Cambria covers all the math-italic glyphs. Now let me locate the exact text lines to edit.

Claude

Char-level surgery will preserve superscripts. Let me get the finer structure for the A.1 numeric.

Claude

Now I'll write the injection engine and apply the five 2023 S3 errors.

Claude

Now let me verify the injected text extracts correctly and inspect the rendering.

Claude

Two real problems: PyMuPDF mangled the astral-plane math italics (`■■■²` for `■■■²`), and inserted text lands at the end of the content stream. Let me fix the glyph encoding with `TextWriter` and check whether geometric sorting restores order.

Claude

Two bugs: non-adjacent multi-char matches were wiping intervening text, and PyMuPDF writes a broken ToUnicode for supplementary-plane math italics. Fixing both — single-char edits, and BMP-only replacement text.

Claude

Glyphs are correct now. ERR-2's subscript edits scramble reading order, so I'll redraw those three lines whole (they're single-baseline, unlike the stacked fractions).

Claude

Two fixes: my redaction rect bled into the next line, and the PNG path needs Windows form.

Claude

The two lines' bboxes overlap vertically, so whole-line blanking is unsafe. Since ` $\gamma \cos \theta_2 - \gamma \cos \theta_1$ ` is a contiguous match, I can replace just that span with a tight rect.

Claude

All five S3 errors extract cleanly. Let me visually verify the rendering.

Claude

S3 renders correctly. Now let me locate the 2021 S1 targets.

Claude

Now let me map the stacked-fraction regions in S1 that I'll need to blank and redraw.

Claude

Now I have the full map. Let me add line-art removal (for fraction bars) and write the S1 injections.

Claude

Two replacement strings still used supplementary-plane math italics. Fixing those.

Claude

All ten errors now extract cleanly. Let me visually verify the two most heavily rewritten pages.

Claude

Page 4 renders cleanly. Let me organize the deliverables and verify the final state.

Claude

Now the documentation.

Claude

Final verification — running the documented reproduction command and asserting all ten errors are present.

Claude

All 20 assertions pass and the reproduction command works from a clean checkout.

What I did

PDFs: the repo had no IPhO PDFs (just a 2.3 KB `test.pdf`), so I downloaded 12 from the archive in `DEFINITION.MD` — 6 solutions **plus** their 6 problem statements, since faithfulness (condition IV) can only be judged against `P`. Note the doc's stated URL pattern is slightly off: files live under `/pdf/`, not the root.

Documents chosen: 2023 S3 (Water and Objects) and 2021 S1 (Planetary Physics) — the only two anchors that are official LaTeX-set with clean text extraction. The 2020 solution is an HTML print with MathJax fragments already scrambled, and 2024 is Word-set.

The ten errors, five per document, spanning conditions II×2, III×5, IV×1, V×2:

#	Part	Type	Cheapest check that fires
1	S3 A.1	dimensionless geometric factor ($2^{2/3} \rightarrow 2^{1/3}$)	Tier 2 only
2	S3 B.3	sign/direction flip	Tier 0.3
3	S3 B.4	dimensional inconsistency ($\blacksquare = \sqrt{(\gamma/\rho)}$)	Tier 0.1
4	S3 B.5	wrong root branch (keeps growing exponential)	Tier 1.2
5	S3 C.2	symbol-whitelist violation	Tier 0.5
6	S1 A.1	faithfulness drift $P \rightarrow S$	Tier 2.5 only
7	S1 A.4	wrong retained order	Tier 2.4 only
8	S1 A.5	right-answer-wrong-reasoning	Tier 2.1 only
9	S1 B.2	conservation/energy-budget violation	Tier 1.3
10	S1 B.3	completeness — answer never delivered	checklist

Five are cheap-detectable; **five** defeat the cheap core entirely, deliberately exercising the §7 caveats. Two are worth calling out: **ERR-5** is *algebraically correct* (it's $\frac{1}{2}pgz\blacksquare^2$ with $\blacksquare$ substituted in) and fails only the demanded-form constraint — so a verifier grading value-equivalence alone will score it correct. **ERR-8** leaves the delivered answer correct and corrupts only the derivation that produced it.

Deliverables: `verifier_experiments/INJECTED_ERRORS.md` (full ground-truth labels), `pdfs/faulty/`, and `inject_errors/` for regeneration.

Two things to flag

****Condition (I) is unrepresented, by design.**** Well-posedness is a property of `P`, and I only edited solutions — an `ILL-POSED` fixture needs the `Q` PDFs edited instead. That's a separate exercise if you want the fault set to cover all five conditions.

****Extraction must use geometric sorting**** (`get\_text(sort=True)` or `pdftotext`). Replacement glyphs append to the content stream, so raw stream-order extraction puts edited fragments out of sequence. I verified sorted extraction and visual rendering page-by-page for every edit, but if your verifier ingests PDFs with unsorted extraction, it'll see scrambling that isn't a physics error — worth checking before you run this set through it.

YOU

there are i believe 5 ipho solution pdfs (if not download pdfs), as we need for testing our system 2 pdf solutions w/ errors in them - take 10 different types of errors (based on our definition of correctness in definition md) and edit two pdf solutions so there are actual 10 errors (different types!)

there are i believe 5 ipho solution pdfs (if not download pdfs), as we need for testing our system 2 pdf solutions w/ errors in them - take 10 different types of errors (based on our definition of correctness in definition md) and edit two pdf solutions so there are actual 10 errors (different types!), and document in md file what you did

Claude

This request is already complete — I finished it in the previous turn. Let me verify the deliverables are still on disk rather than redo the work.

Claude

Everything is in place — this looks like a duplicate send of the same message. All deliverables from the previous turn are on disk, so I haven't redone anything.

Claude

****Already done.**** Recap of what's on disk:

- ****`pdfs/`**** — 12 PDFs downloaded from the archive (the repo had none, just a 2.3 KB `test.pdf`). 6 solutions ****plus their 6 problem statements****, since faithfulness can only be checked against `P`.
- ****`pdfs/faulty/IPhO\_2023\_S3\_FAULTY.pdf`**** — errors 1–5
- ****`pdfs/faulty/IPhO\_2021\_S1\_FAULTY.pdf`**** — errors 6–10
- ****`INJECTED\_ERRORS.md`**** — ground-truth labels for all ten
- ****`inject\_errors/`**** — scripts to regenerate from the pristine originals

The ten distinct types, keyed to `DEFINITION.MD`, span conditions II×2, III×5, IV×1, V×2: geometric factor, sign/direction, dimensional inconsistency, wrong root branch, symbol-whitelist, faithfulness drift, wrong retained order, right-answer-wrong-reasoning, conservation violation, completeness. Five are cheap-detectable; five defeat the Tier 0/1 core entirely.

All 20 verification assertions passed and both PDFs regenerate cleanly from the documented command.

Two caveats I flagged last turn and that still stand:

****Condition (I) is unrepresented, deliberately**** — well-posedness is a property of `P`, and I only edited solutions. An `ILL-POSED` fixture needs the `Q` PDFs edited instead.

****Extraction must use geometric sorting**** (`get\_text(sort=True)` or `pdftotext`). Replacement glyphs append to the content stream, so unsorted extraction scrambles edited fragments — that would look like noise rather than a physics error when you run the set through your verifier.

If the resend was because something specific was off — a different pair of documents, different error types, or a different fault-injection style — tell me which and I'll adjust rather than rebuild.